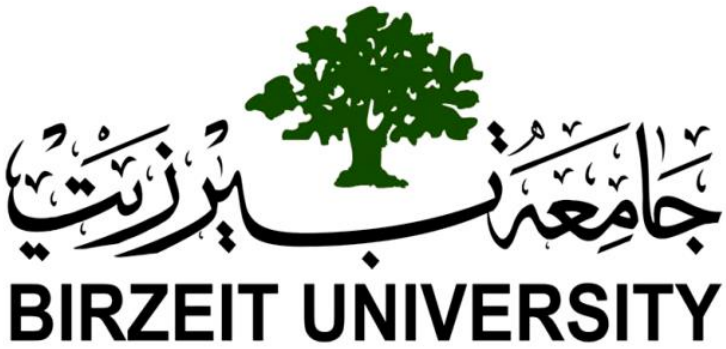


Faculty of Engineering & Technology

Department of Electrical and Computer Engineering

Computer Architecture

ENCS4370

Project#2

Design and Verification of a Simple Pipelined RISC Processor in Verilog

---

Prepared by:

Anas Dawoud	**ID: 1220523**
Abdulrahman Atyani	**ID: 1221808**
Mohammad Abu Hijleh	**ID: 1221350**

Instructor: **Dr. Aziz Qaroush**

Sections: **3&1**
Date: **10/6/2025**

Guidelines for the Course Project Report

Team Member Name	Team Member ID	Contributions
Mohammad Abu Hijleh	1221350	*Block diagram of Processor's Pipelined Datapath *Report Contribution *Testing Contribution *Verilog code contribution
Anas Barghothi	1220523	*Writing Main module Verilog code implementation *Testing Contribution *Main module Design Contribution * Report Contribution
Abdulrahman Atyani	1221808	*Testing Contribution *Report Contribution *Verilog code contribution

Processor Implementation (Tick One)	
Single Cycle	
Multi Cycle	
Pipelined	✅

In case of pipeline implementation (Tick the correct answer)		
	Implemented in RTL Code?	**Verified and Correctly Worked?**
Data hazards detection	✅	✅
Control hazards detection	✅	✅
Structural hazards detection	✅	✅
Forwarding	✅	✅
Stalling	✅	✅

Tick the correct answer

Instruction	**Did you implement this instruction in the RTL?**		**Did you write the verification code for this instruction?**		**Did the instruction Work perfectly when it has been verified?**	
OR Rd, Rs, Rt	Yes	No	Yes	No	Yes	No
ADD Rd, Rs, Rt	✅		✅		✅	
SUB Rd, Rs, Rt	✅		✅		✅	
CMP Rd, Rs, Rt	✅		✅		✅	
ORI Rd, Rs, Imm	✅		✅		✅	
ADDI Rd, Rs, Imm	✅		✅		✅	
LW Rd, Imm(Rs)	✅		✅		✅	

LDW Rd, Imm(Rs)	✅		✅		✅	
SDW Rd, Imm(Rs)	✅		✅		✅	
BZ Rs, Label	✅		✅		✅	
BGZ Rs, Label	✅		✅		✅	
BLZ Rs, Label	✅		✅		✅	
JR Rs	✅		✅		✅	
J Label	✅		✅		✅	
CALL Label	✅		✅		✅	
Tick one of the following						
My processor can execute only test programs consisting of one instruction only						
My processor can execute complete programs (A simulation screenshot must be provided as evidence)						✅

Table 1: Report table

Abstract

This project involves creating and testing a basic 32-bit pipelined RISC processor using Verilog. The processor follows a five-stage pipeline design, including fetch, decode, execute, memory, and write-back stages. It supports a simplified instruction set that handles arithmetic, logic, memory access, and branching instructions, all within a 32-bit format. The system features 16 general-purpose registers, with R15 acting as the program counter and R14 as the return address register. Control logic was carefully developed using state diagrams, truth tables, and Boolean logic. To confirm correct operation, we built a simulation environment and ran multiple test programs written in the processor's instruction set.

Table of contents

Guidelines for the Course Project Report 2

Abstract 4

Table of contents 5

Table of Figures 8

List of Tables 9

Datapath description, components, and implementation details 10

1. Fetch Stage (IF) 10

Components: 10

Assembly: 12

2. Decode Stage (ID) 12

Components: 12

Assembly: 14

3. Execute Stage (EX) 14

Components: 14

Assembly: 14

4. Memory Stage (MEM) 15

Components: 15

Assembly: 15

5. Write-Back Stage (WB) 16

Components: 16

Assembly: 16

Pipeline Registers 16

Implementation Details and Design Choices 18

1. Hazard Resolution 18

2. Control Signal Management 19

3. LDW/SDW Handling 20

4. Branch/Jump Handling 20

5. Call (CLL) Implementation 21

6. Pipeline Register Design 21

Datapath Block Diagram 22

Control signals 23
Pc control unit: 23
Enable 23
Enable2 23
Pc_sel 24
IF_reg_in_sel 24
Main control 26
ALUOP: 26
MEMO_R 26
MEMO_W 26
RW 26
ALU_SOR 27
WB 27
CLL 27
Hazzard: 28
ALU_sor_sel1 28
ALU_sor_sel2 28
Rd_haz 28
Test cases: 29
OR 29
ADD 30
SUB 31
CMP 32
ORI 35
ADDI 36
LW 37
SW 38
LDW 39
SDW 40
BZ 41
BGZ 43
BLZ 45

JR 47
J 48
CLL 49
Code example 51
Code 1: 51
Code 2: 52
Code 3: 53
Code 4: 54
RTL Pipeline Design Description 55
R-Type Instructions 55
(OR, ADD, SUB, CMP — Opcodes: 0 to 3) 55
I-Type Instructions 55
(ORI, ADDI — Opcodes: 4, 5) 55
Load/Store Instructions 56
LW Rd, Imm (Rs) — Opcode: 6 56
SW Rd, Imm(Rs) — Opcode: 7 56
Double-Word Instructions 57
LDW Rd, Imm(Rs) — Opcode: 8 57
SDW Rd, Imm(Rs) — Opcode: 9 57
Branch Instructions 58
BZ Rs, Label — Opcode: 10 58
BGZ Rs, Label — Opcode: 11 58
BLZ Rs, Label — Opcode: 12 58
Jump Instructions 59
JR Rs — Opcode: 13 59
J Label — Opcode: 14 59
CLL Label — Opcode: 15 59
Conclusion 60
Future Improvements 60

Table of Figures

Figure 1: Program Counter module 10
Figure 2: Instruction Memory module 10
Figure 3: 4:1 PC Mux 11
Figure 4: PC Control Unit module 11
Figure 5: Register File module 12
Figure 6: ID Control Unit module 12
Figure 7: Hazard Handling Control Unit 13
Figure 8: LDW/SDW modules 13
Figure 9: ALU module 14
Figure 10: Data Memory module 15
Figure 11: Write-Back Mux (2:1) 16
Figure 12: IF/ID Register 16
Figure 13: ID/EX Register 17
Figure 14: EX/MEM Register 17
Figure 15: MEM/WB Register 18
Figure 16: Forwarding Logic 18
Figure 17: Stalling Logic 19
Figure 18: Control Signal Management 20
Figure 19: Branch/Jump Handling 21
Figure 20: enable222 Logic 21
Figure 21: enable22 Logic 21
Figure 22: Datapath Block Diagram 22
Figure 23: EX OR 29
Figure 24: EX ADD 30
Figure 25: EX SUB 31
Figure 26: EX CMP1 32
Figure 27: EX CMP2 33
Figure 28: EX CMP3 34
Figure 29: EX ORI 35
Figure 30: EX ADDI 36
Figure 31: EX LW 37
Figure 32: EX SW 38
Figure 33: EX LDW 39
Figure 34: EX SDW 40
Figure 35: EX BZ instruction memo 41
Figure 36: EX BZ branch 41
Figure 37: EX BZ no branch 42
Figure 38: EX BGZ instruction memo 43

Figure 39: EX BGZ branch 43
Figure 40: EX BGZ no branch 44
Figure 41: EX BLZ instruction memo 45
Figure 42: EX BLZ branch 45
Figure 43: EX BLZ no branch 46
Figure 44: EX JR instruction memo 47
Figure 45: EX JR 47
Figure 46: EX J instruction memo 48
Figure 47: EX J 48
Figure 48: EX CLL instruction memo 49
Figure 49: EX CLL 49
Figure 50: CODE 1 51
Figure 51: CODE 2 52
Figure 52: CODE 3 53

List of Tables

Table 1: Report table 3
Table 2: PC control unit signals truth table 25
Table 3: Instructions signals truth table 26

Datapath description, components, and implementation details

The pipelined RISC processor Datapath is organized into five classic stages: **Fetch (IF)**, **Decode (ID)**, **Execute (EX)**, **Memory (MEM)**, and **Write-Back (WB)**. The design integrates hazard handling, forwarding, and specialized control logic to support the ISA requirements. Below is a breakdown of the components and their integration:

1. Fetch Stage (IF)

Components:

- **Program Counter (PC):** 32-bit register holding the next instruction address.

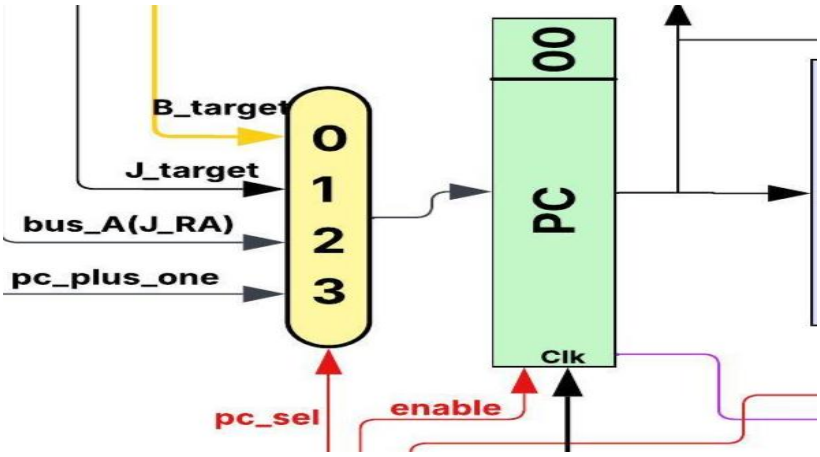


Figure 1: Program Counter module

- **Instruction Memory:** 256-word ROM storing instructions (word-addressable).

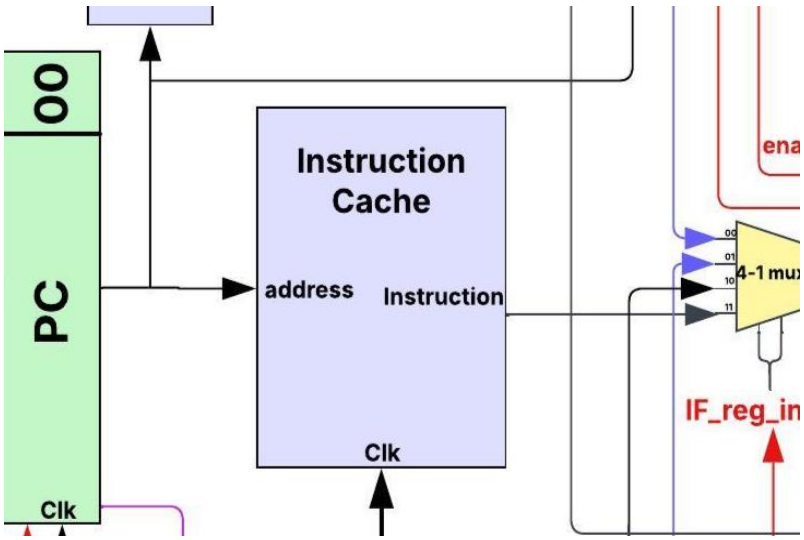


Figure 2: Instruction Memory module

- **PC Adder:** Computes PC + 1 for sequential execution.
- **Branch/Jump Units:**
    - Branch: Calculates branch target as PC + sign-extended Imm.
    - Jump: Computes absolute jump target from Imm.
- **4:1 PC Mux:** Selects next PC from:
    - 00: Branch target
    - 01: Jump target
    - 10: Register-based jump (bus_A = Rs)
    - 11: PC + 1 (default)

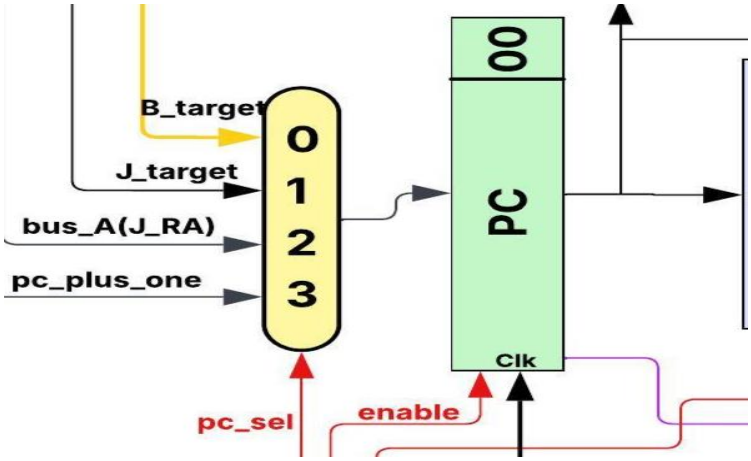


Figure 3: 4:1 PC Mux

- **PC Control Unit:** Generates pc_sel and handles stalls for LDW/SDW.

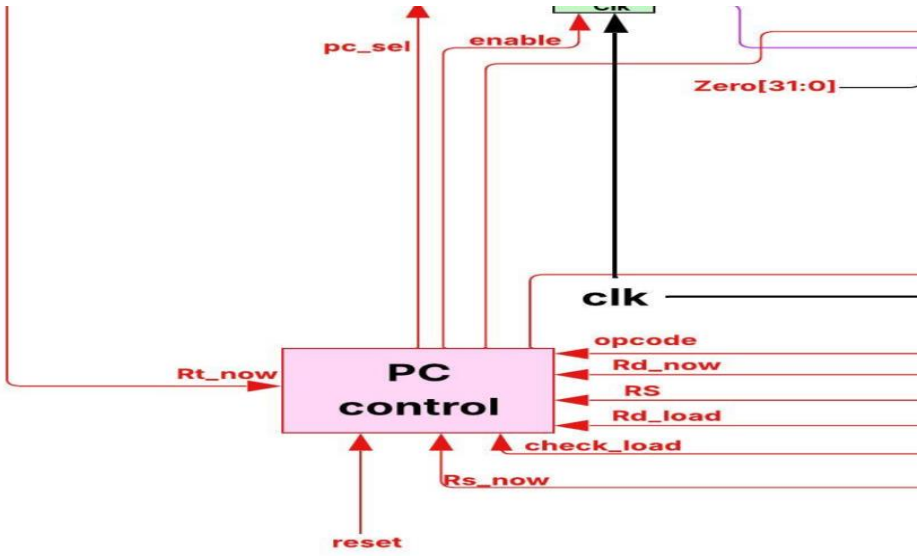


Figure 4: PC Control Unit module

Assembly:

The stage fetches instructions using the PC value. The PC Mux redirects flow for branches/jumps, while the PC Control unit manages stalls (e.g., for LDW/SDW) by gating enable signals. The output (instruction, npc) passes to the **IF/ID pipeline register**.

2. Decode Stage (ID)

Components:

- **Register File:** 16x32-bit registers (R0–R15). R14 stores return addresses; R15 is PC.

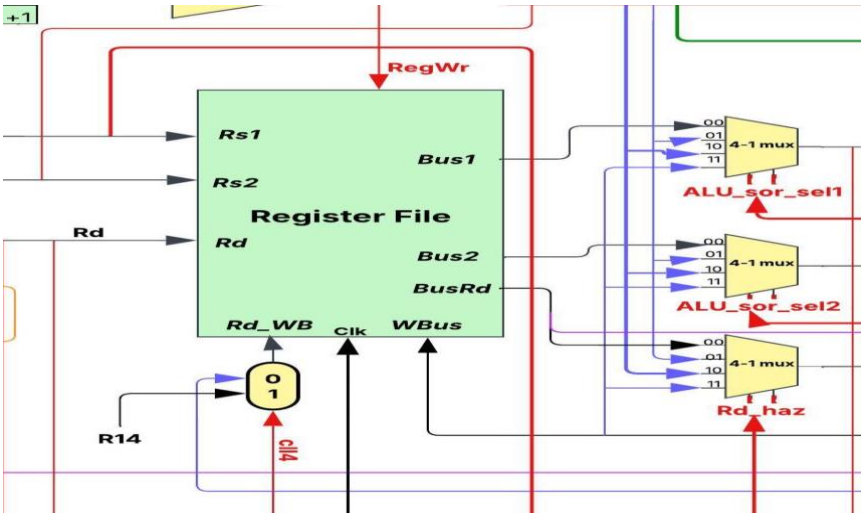


Figure 5: Register File module

- **Control Unit:** Decodes opcode to generate signals:
    - WB, memo_r/w, RW, ALU_sor, ALUop, cll (call).



Figure 6: ID Control Unit module

- **Sign Extender:** Extends 14-bit Imm to 32 bits (sign/zero-based).

- **Hazard Handling:**
    - **Forwarding Unit:** Resolves RAW hazards by selecting data from EX/MEM/WB stages via 4:1 muxes for Rs, Rt, Rd.
    - **Stall Logic:** Detects LDW/SDW and load-use hazards, injects NOP via IF_reg_in_sel.

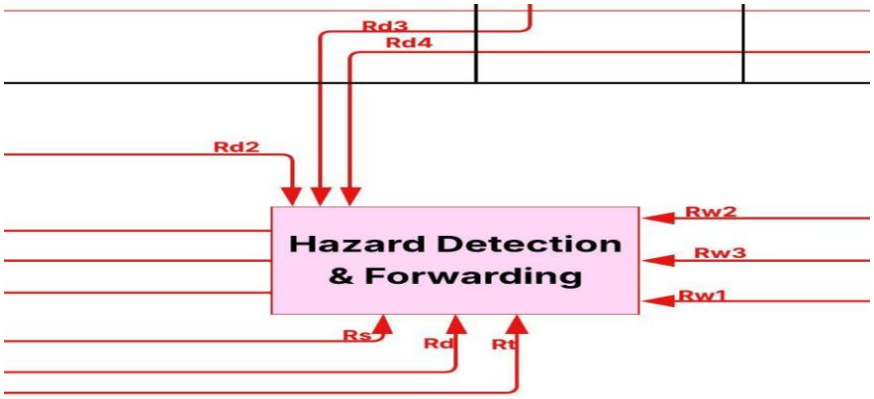


Figure 7: Hazard Handling Control Unit

- **LDW/SDW Handler:** Splits double-word ops into micro-operations (e.g., LW Rd, Imm(Rs) → LW Rd+1, Imm+1(Rs)).

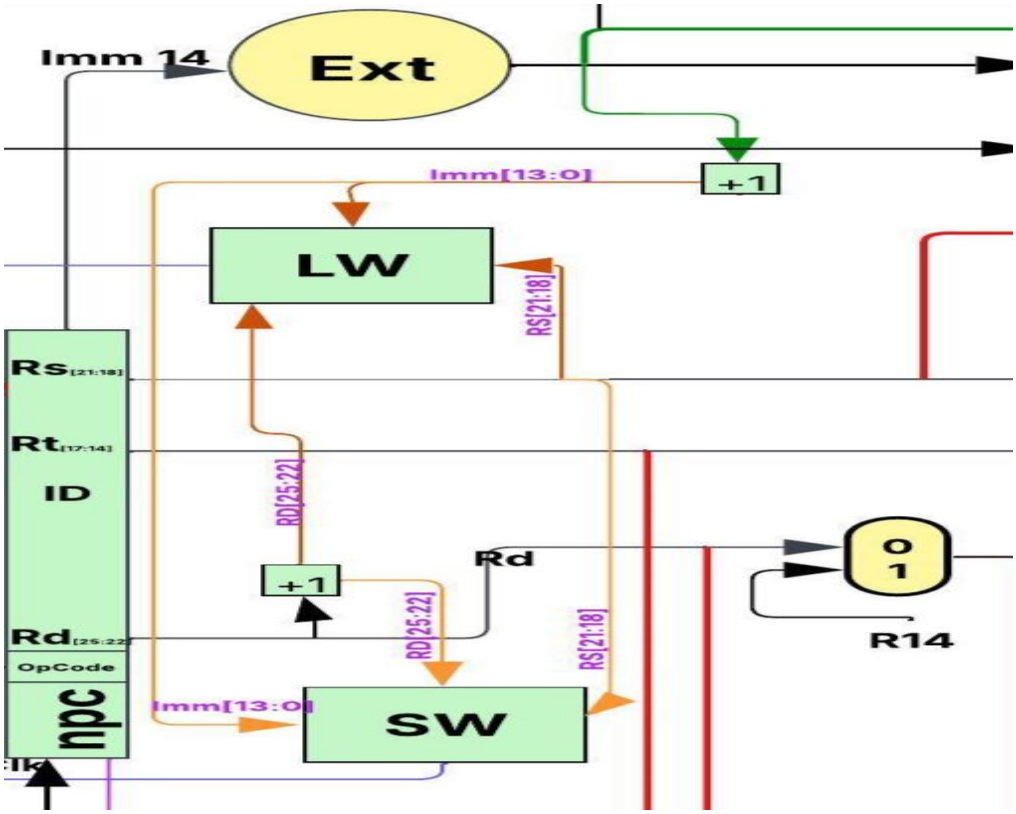


Figure 8: LDW/SDW modules

Assembly:

Register values (Bus1, Bus2, BusRd) are read and forwarded as needed. Control signals and extended Imm are stored in the **ID/EX pipeline register** alongside A, B, R, and Rd values. The kill_reg injects zeros during flushes.

3. Execute Stage (EX)

Components:

- **ALU:** Supports:
    - 00: OR | 01: ADD | 10: SUB | 11: CMP (outputs 0, 1, or -1).

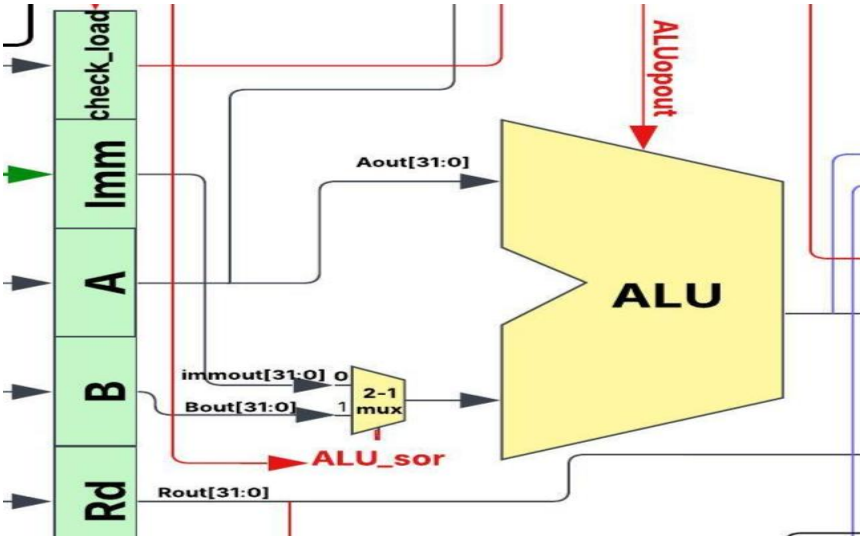


Figure 9: ALU module

- **ALU Source Mux (2:1):** Selects between Bout (register) or immout (immediate).
- **Branch Condition Logic:** Evaluates Rs (forwarded) for BZ/BGZ/BLZ in ID stage.

Assembly:

The ALU executes operations using Aout and the selected source operand. Results (ALU_result, Rd) pass to the **EX/MEM pipeline register**. The cll signal routes npc (return address) to R14.

4. Memory Stage (MEM)

Components:

- **Data Memory:** 256-word RAM (word-addressable).

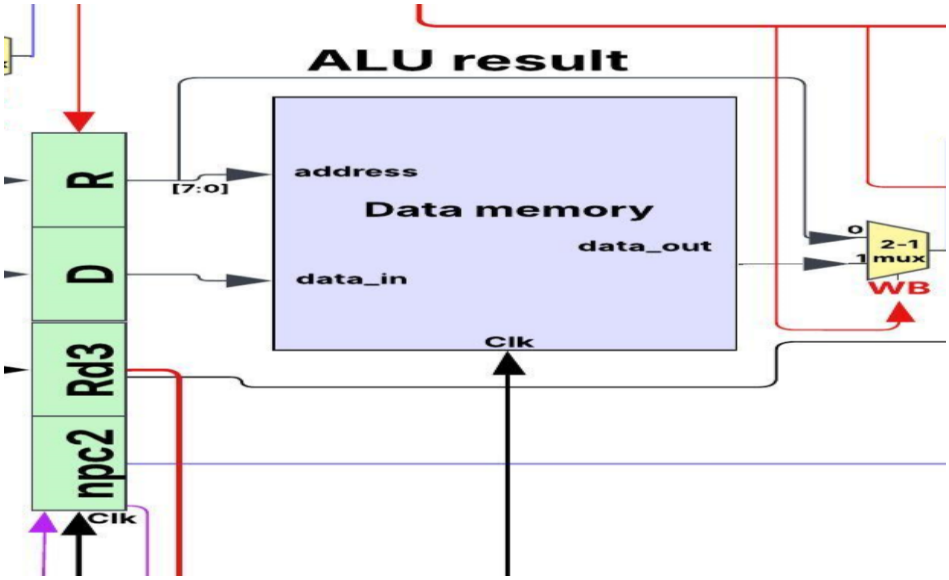


Figure 10: Data Memory module

- **Memory Mux (2:1):** Selects write_back_data from ALU result or memory read.

Assembly:

On memWrite/memRead, data is written/read at ALU_result_out[7:0]. Loads route through a 2:1 mux (ALU result vs. memory). Outputs stored in **MEM/WB pipeline register**.

5. Write-Back Stage (WB)

Components:

- **Write-Back Mux (2:1):** Routes R_in to register file from:
    - WriteBackData_out (MEM stage) or npc (for CLL).

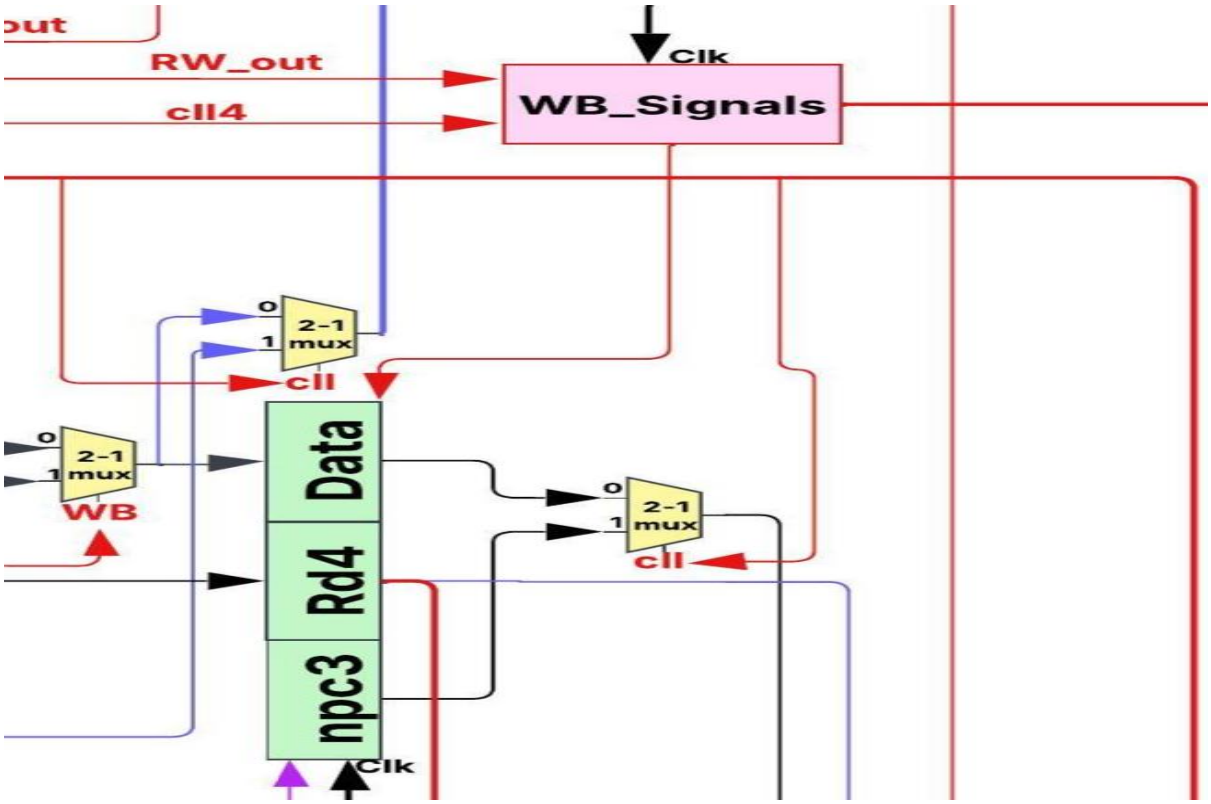


Figure 11: Write-Back Mux (2:1)

Assembly:

Data is written to Rd_WB when RW_out4=1. Forwarding paths feed results back to ID/EX.

Pipeline Registers

- **IF/ID:** Buffers instruction and npc.

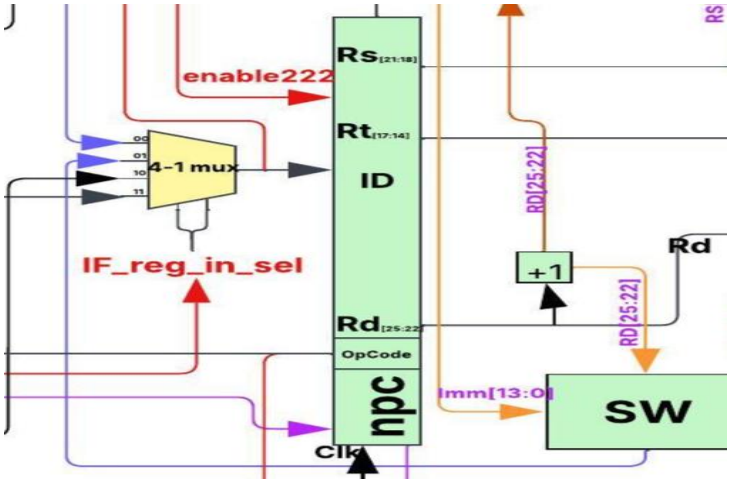


Figure 12: IF/ID Register

- **ID/EX:** Passes control signals, A/B/R values, Imm, Rd, and npc.

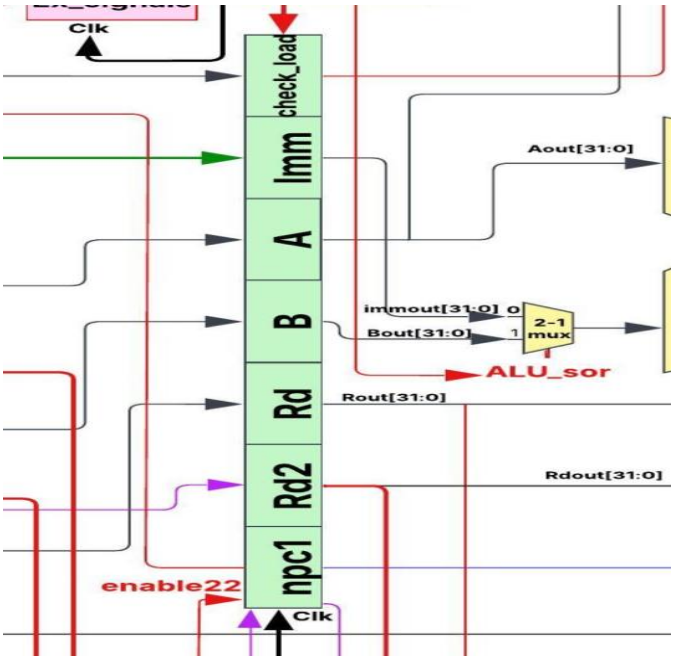


Figure 13: ID/EX Register

- **EX/MEM:** Carries ALU_result, data_in (store data), Rd, and control.

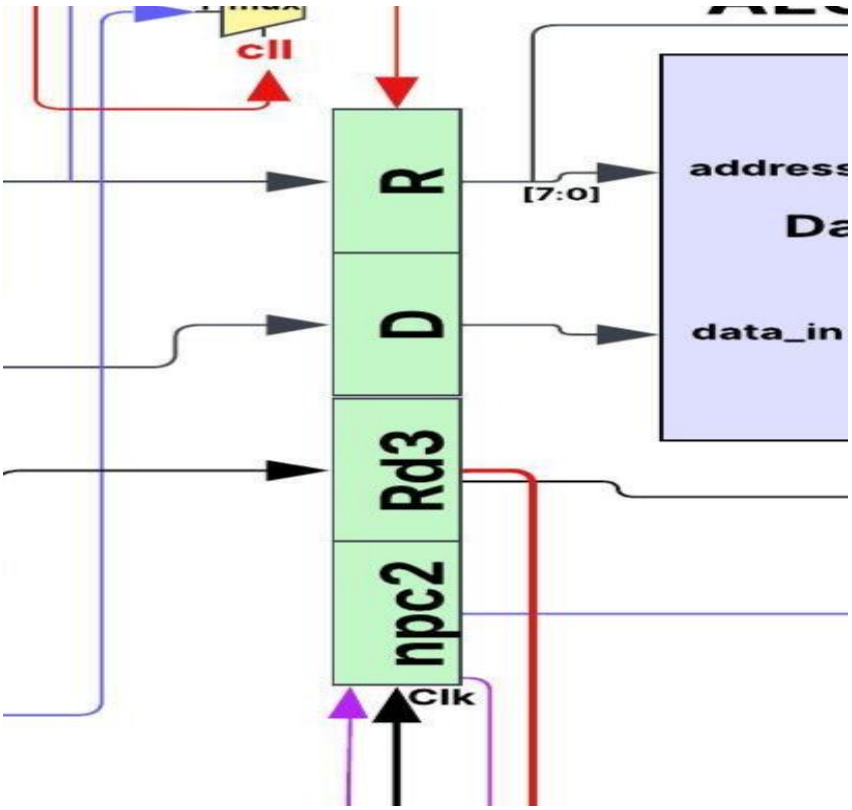


Figure 14: EX/MEM Register

- **MEM/WB:** Preserves WriteBackData, Rd4, and RW signal.

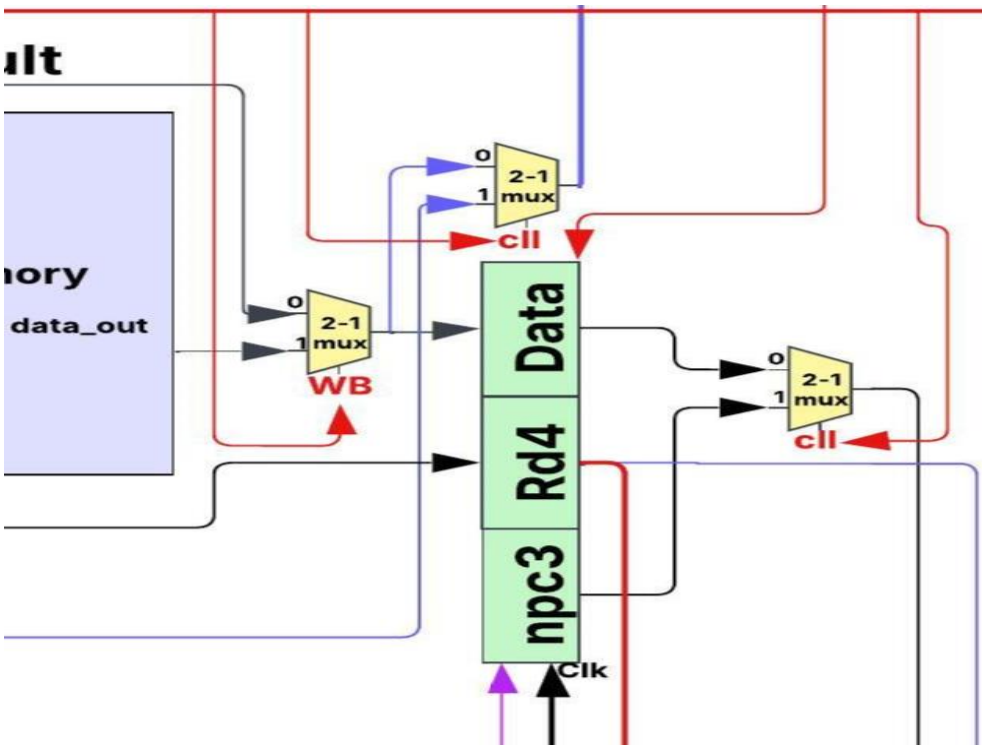


Figure 15: MEM/WB Register

Implementation Details and Design Choices

1. Hazard Resolution

Forwarding

- **Choice:** 4:1 muxes in ID stage forward from EX (h1), MEM (h2), or WB (h3) to Rs/Rt/Rd.

- **Justification:** Eliminates RAW stalls for back-to-back dependencies (e.g., ADD R1, R2, R3 → OR R4, R1, R5).

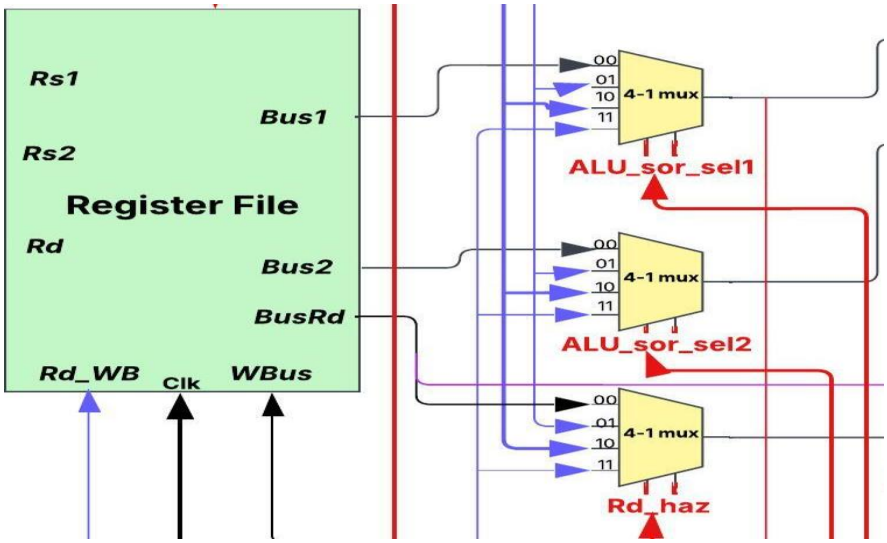


Figure 16: Forwarding Logic

Stalling

- **Choice:** PC Control stalls fetch for LDW/SDW and load-use hazards by:
    - Deasserting enable (freezes PC).
    - Injecting NOP via IF_reg_in_sel (selecting zero or modified instruction).
- **Justification:** Ensures LDW/SDW complete in two cycles without corrupting pipeline flow.

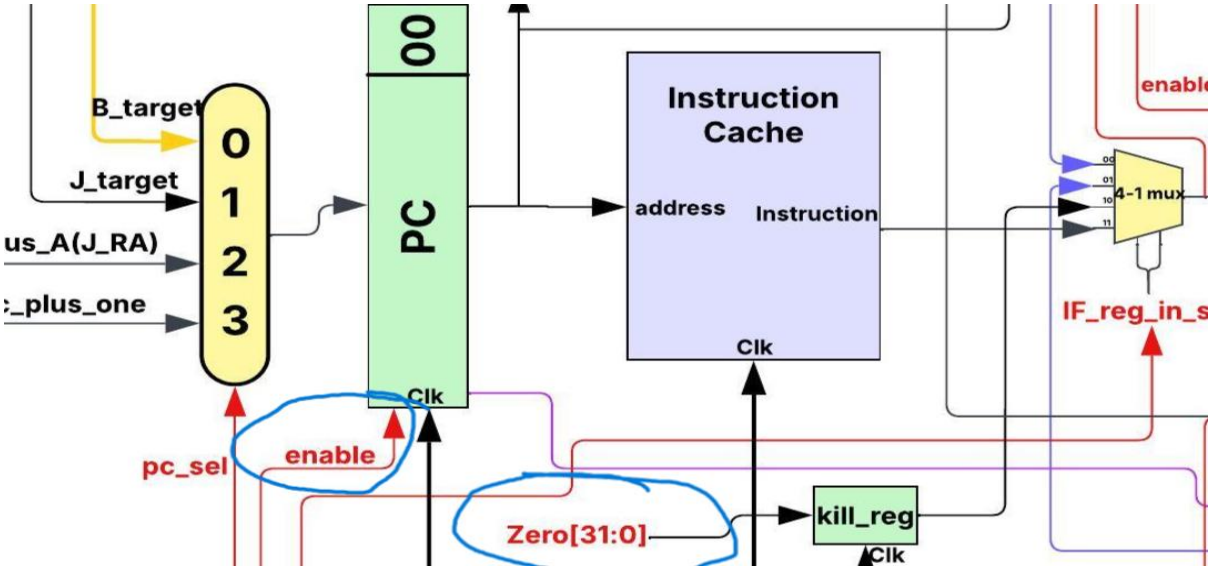


Figure 17: Stalling Logic

2. Control Signal Management

- **Centralized Control:**
    - **Choice:** Control signals generated in ID stage propagate through pipeline registers.
    - **Justification:** Simplifies timing; each stage uses locally buffered signals.
- **Signal Killing:**
    - **Choice:** cll (call) and branch mispredictions flush pipeline via IF_reg_in_sel.
    - **Justification:** Avoids erroneous execution after redirection.

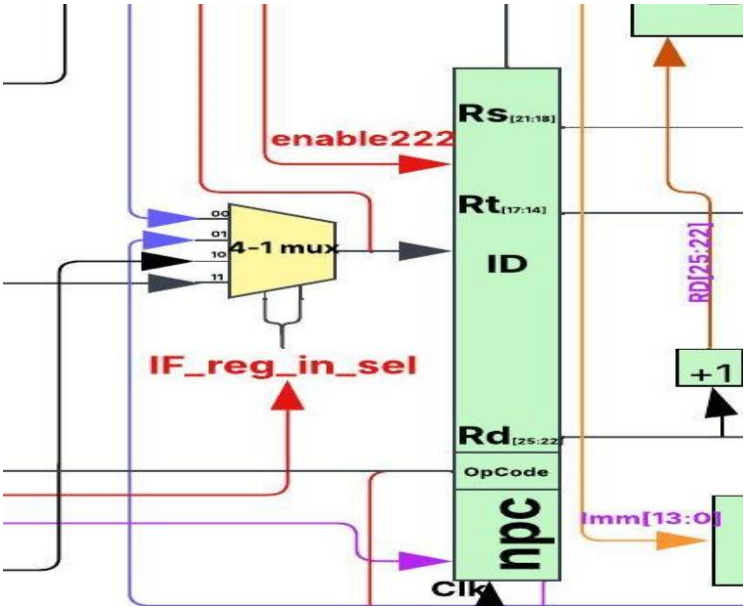


Figure 18: Control Signal Management

3. LDW/SDW Handling

- **Micro-Operation Splitting:**
  - **Choice:** LDW/SDW replaced in ID stage with two sequential LW/SW (e.g., LDW R0, Imm(Rs) → LW R0, Imm(Rs) → LW R1, Imm+1(Rs)).
  - **Justification:** Reuses existing logic without adding complex memory interfaces.
- **Register Pair Enforcement:**
  - **Choice:** LDW checks if Rd is even; odd values trigger exceptions (not fully implemented).
  - **Justification:** Ensures alignment for double-word ops

4. Branch/Jump Handling

- **Early Evaluation:**
  - **Choice:** Branches resolved in ID stage using forwarded Rs value.
  - **Justification:** Reduces branch penalty to 1 cycle (vs 2 in EX stage).
- **Multi-Target PC:**

- **Choice:** 4:1 mux handles branches, jumps, and register-based jumps (JR).
- **Justification:** Simplifies redirection logic while supporting all control-flow instructions.

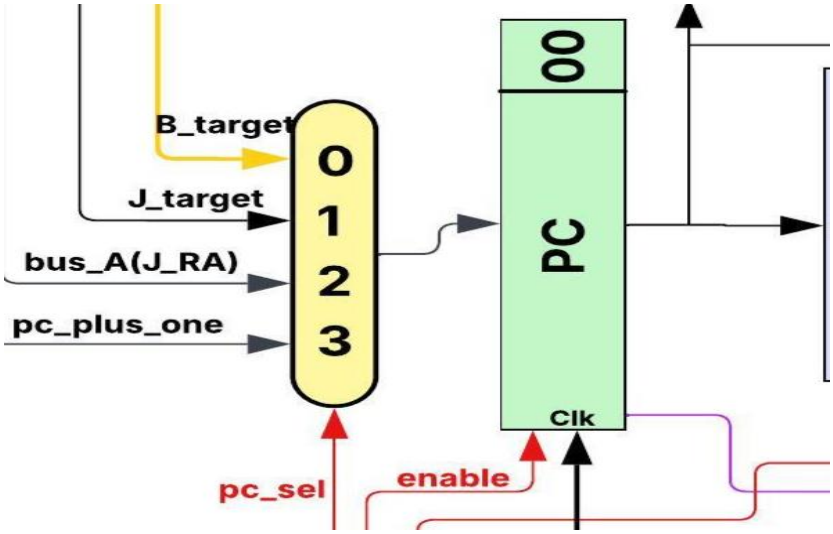


Figure 19: Branch/Jump Handling

5. Call (CLL) Implementation

- **Choice:**
  - CLL routes PC + 1 to R14 via npc path and jumps to Imm.
  - WB stage selects npc for R_in when cll_out=1.
- **Justification:** Uses existing ALU/npc paths instead of dedicated logic.

6. Pipeline Register Design

- **Choice:** Registers include enable signals (enable22, enable222) for stalling.
- **Justification:** Allows freezing stages during hazards without manual flushing.

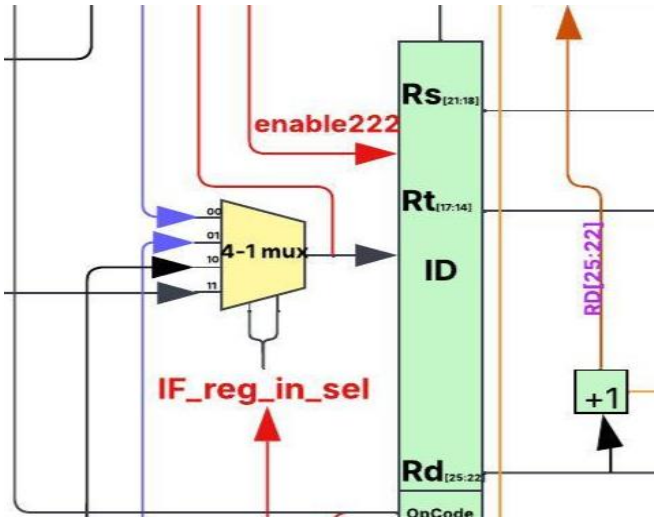


Figure 20: enable222 Logic

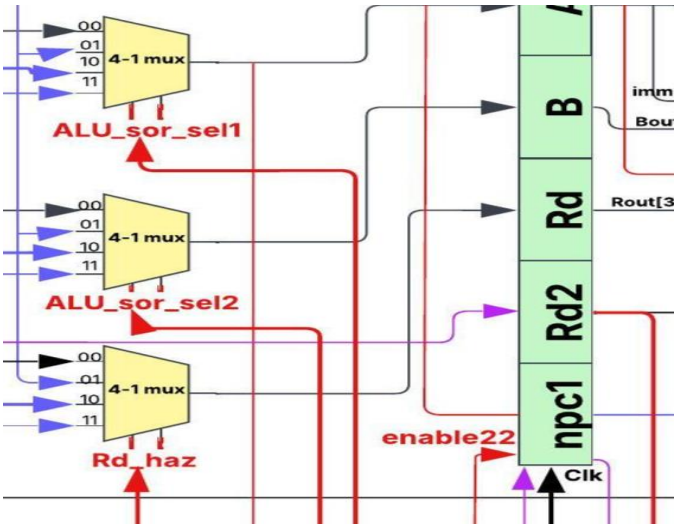


Figure 21: enable22 Logic

Datapath Block Diagram

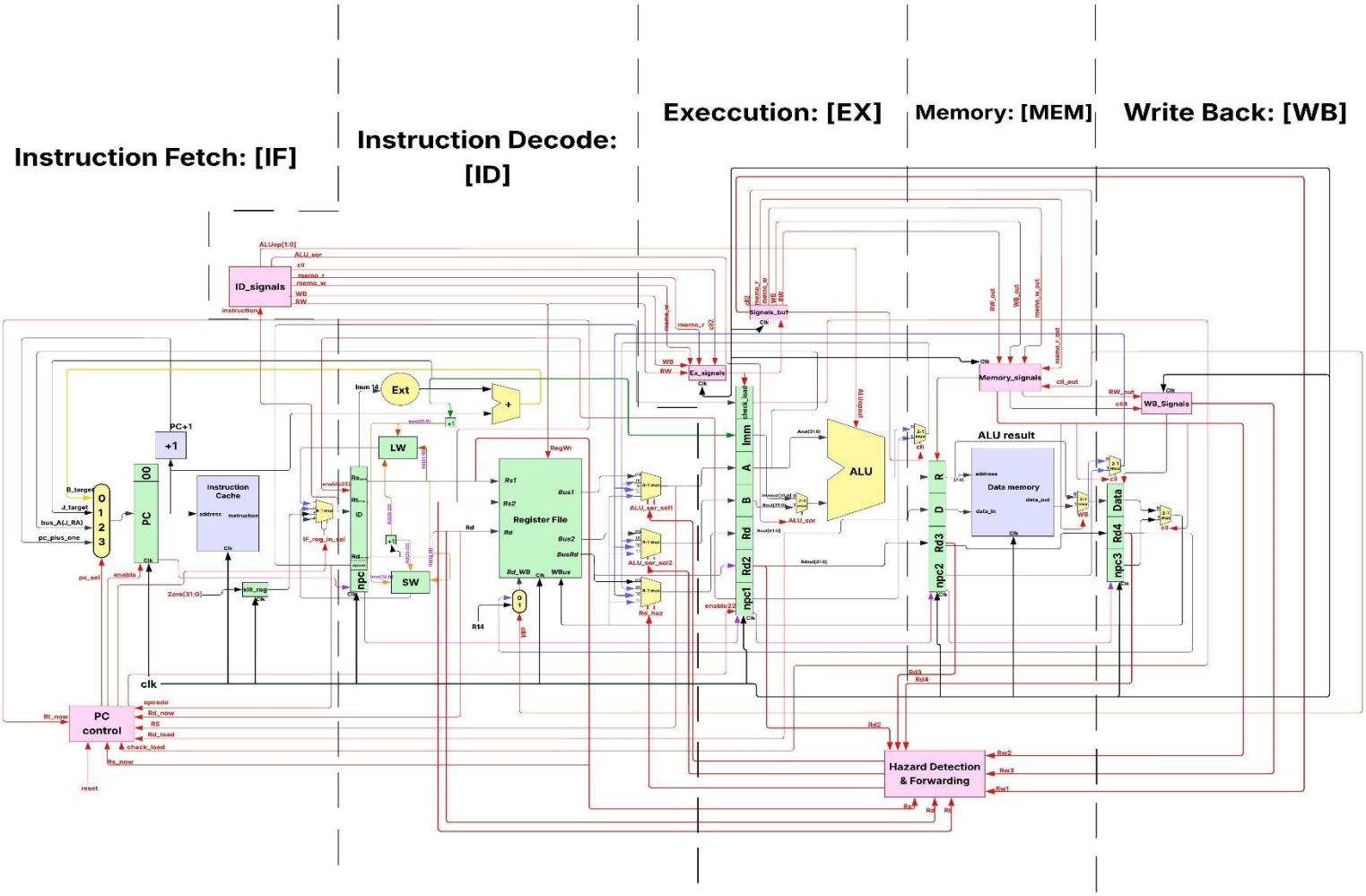


Figure 22: Datapath Block Diagram

Control signals

Pc control unit:

Those signals will be ready after instruction enter ID stage.

Enable

The enable signal controls whether the PC (Program Counter) can be updated. If the enable signal is 1, the PC can be modified; otherwise, it remains unchanged.

In the case of a LW (load word) instruction when it in EX stage, the enable signal might be either 1 or 0. This is because if a dependent instruction immediately follows the LW instruction, it must wait until the LW reaches the Memory stage to avoid using incomplete data. In such cases, the pipeline may need to stall, and the enable signal would be set to 0 to hold the PC and prevent fetching a new instruction. Also it will be zero when LDW and SDW in ID.

Enable = ~(LDW + SDW + (LW in_ex . (Rs_{load} xnor Rs ID). ~ LW ID + Lw in_ex . (Rtload xnor Rt ID). ~ LW ID + SW . (Rd load xnor Rd ID))

Enable2

The enable2 signal determines whether the Instruction Register (IR) can be updated. When enable2 is set to 1, the IR is allowed to be written to; otherwise, it holds its current value.

In the case of a lw (load word) instruction, we need to check for a data hazard with the instruction that follows it. Specifically, when the lw instruction reaches the EX (Execute) stage, we compare it with the instruction currently in the ID (Instruction Decode) stage.

If the instruction in the ID stage depends on the result of the lw (i.e., it needs the value being loaded), then a data hazard exists. To handle this, we introduce a stall cycle by setting enable2 = 0, which prevents the instruction register from updating. This effectively delays the next instruction until the data from the lw is available, resolving the hazard.

Enable2 = ~(LW in_ex . (Rs_{load} xnor Rs ID). ~ LW ID + Lw in_ex . (Rtload xnor Rt ID). ~ LW ID + SW . (Rd load xnor Rd ID))

Pc_sel

pc_sel is the selector for the mux that use to update the pc value:

11 → pc +1

10 → pc = Rs

01 → pc = imm

00 → pc =pc +imm

Logic:

If ((BZ . T) + (BGZ . T) + (BLZ . T)) Pc_sel =00.

Else if (JR) pc_sel = 10.

Else if (J + CLL) Pc_sel = 01.

Else pc_sel =11.

IF_reg_in_sel

The ID_reg_in_sel is the selector use for the mux that enter the instruction in ID reg

11 → instruction memo(pc)

10 → kill 000000000

01 → Sw Rd+1 ,Rs,imm+1.

00 → lw Rd+1 Rs, imm+1.

Logic:

If ((BZ . T) + (BGZ . T) + (BLZ . T) + JR + J + CLL) Pc_sel =10.

Else if (SDW) pc_sel = 01.

Else if (LDW) Pc_sel = 00.

Else pc_sel =11.

Instruction	Enable	Enable2	Pc_sel	IF_reg_in_sel
OR	1	1	11	11
ADD	1	1	11	11
SUB	1	1	11	11
CMP	1	1	11	11
ORI	1	1	11	11
ADDI	1	1	11	11
LW	0 or 1	0 or 1	11	11
SW	1	1	11	11
LDW	0	1	11	00
SDW	0	1	11	01
BZ	1	1	00 or 11	10 or 11
BGZ	1	1	00 or 11	10 or 11
BLZ	1	1	00 or 11	10 or 11
JR	1	1	10	10
J	1	1	01	10
CLL	1	1	01	10

Table 2: PC control unit signals truth table

Main control

Instruction	ALUOP	MEMO_W	MEMO_R	RW	ALU_SOR	WB	CLL
OR	00	0	0	1	0	0	0
ADD	01	0	0	1	0	0	0
SUB	10	0	0	1	0	0	0
CMP	11	0	0	1	0	0	0
ORI	00	0	0	1	1	0	0
ADDI	01	0	0	1	1	0	0
LW	01	0	1	1	1	1	0
SW	01	1	0	0	1	X	0
LDW	01	0	1	1	1	1	0
SDW	01	1	0	0	1	X	0
BZ	XX	0	0	0	X	X	0
BGZ	XX	0	0	0	X	X	0
BLZ	XX	0	0	0	X	X	0
JR	XX	0	0	0	X	X	0
J	XX	0	0	0	X	X	0
CLL	XX	0	0	0	X	X	1

Table 3: Instructions signals truth table

ALUOP:

If (OR + ORI) ALUOP = 00.

Else if (ADD +ADDI + LW + SW + LDW + SWD) ALUOP =01.

Else if (SUB) ALUOP =10.

Else if (CMP) ALUOP =11.

Else ALUOP =XX.

MEMO_R = LW + LDW

MEMO_W = SW + SDW

RW = OR +ORI + ADD + ADDI + CMP + SUB +LW + LDW.

ALU_SOR = ORI + ADDI +LW+SW+LDW+SWD.

WB = LW + LDW

CLL =CLL

Hazzard:

ALU_sor_sel1

if (Rd2 == Rs && RW1) ALU_sor_sel1 = 01;

else if (Rd3 == Rs && RW2) ALU_sor_sel1 = 1

else if (Rd4 == Rs && RW3) ALU_sor_sel1 = 11

else ALU_sor_sel1 = 00

ALU_sor_sel2

if (Rd2 == Rt && RW1) ALU_sor_sel2 = 01

else if (Rd3 == Rt && RW2) ALU_sor_sel2 = 10

else if (Rd4 == Rt && RW3) ALU_sor_sel2 = 11

else ALU_sor_sel2 = 00

Rd_haz

if (Rd2 == Rd && RW1) Rd_haz = 01

else if (Rd3 == Rd && RW2) Rd_haz = 10

else if (Rd4 == Rd && RW3) Rd_haz = 11

else Rd_haz = 00

Test cases:

OR

OR R2, R0, R1.

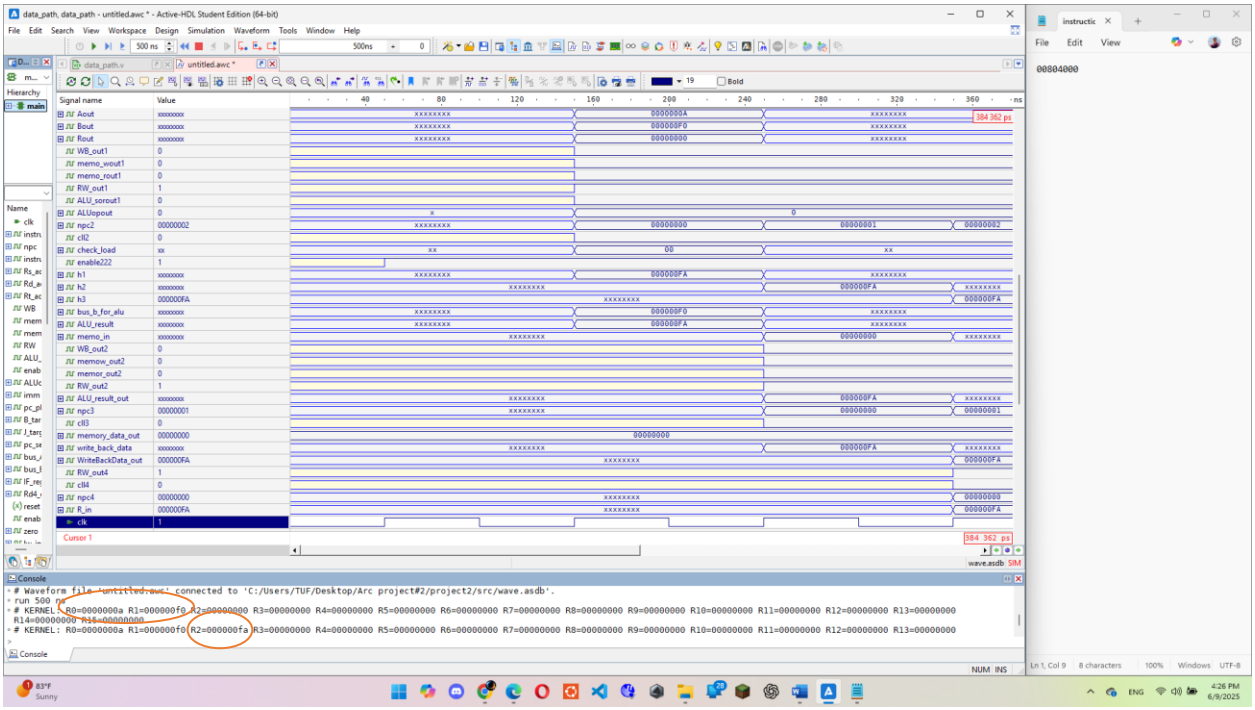


Figure 23: EX OR

An OR instruction that performs a bitwise OR operation between R1 and R0, and stores the result in R2.

ADD

ADD R2, R0, R1

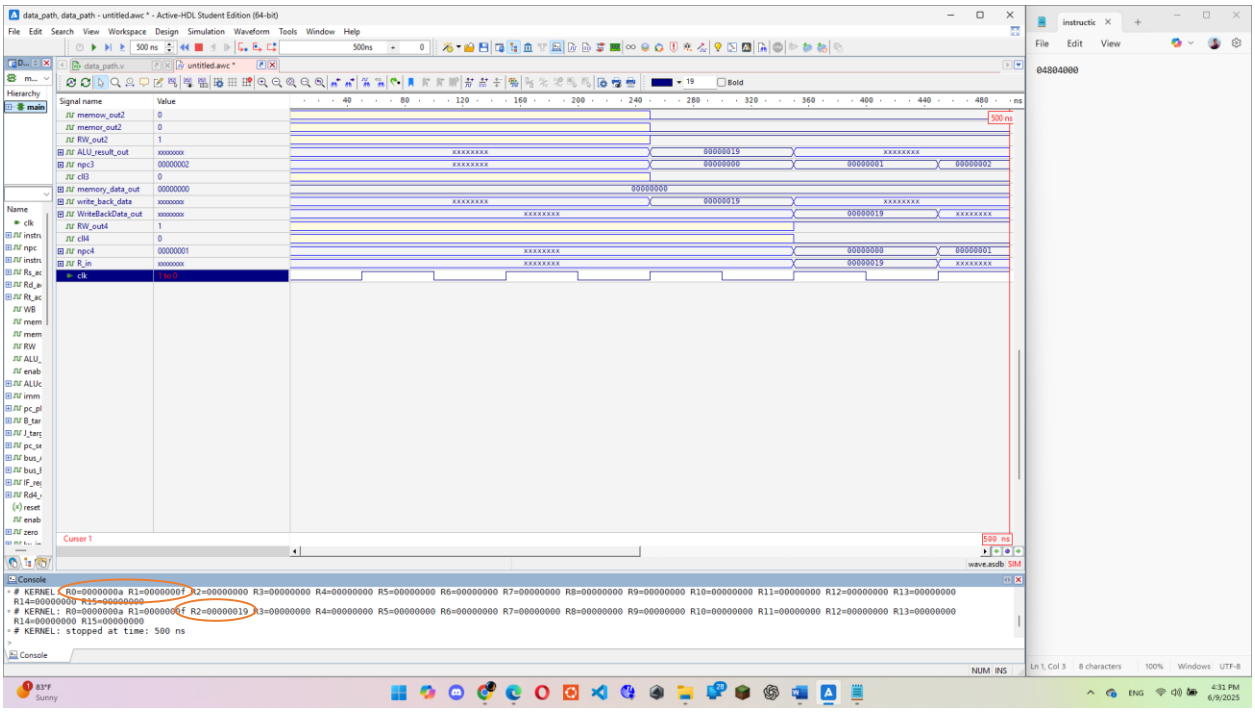


Figure 24: EX ADD

An ADD instruction that performs an addition of R0 and R1, and stores the result in R2.

SUB

SUB R2,R1,R0

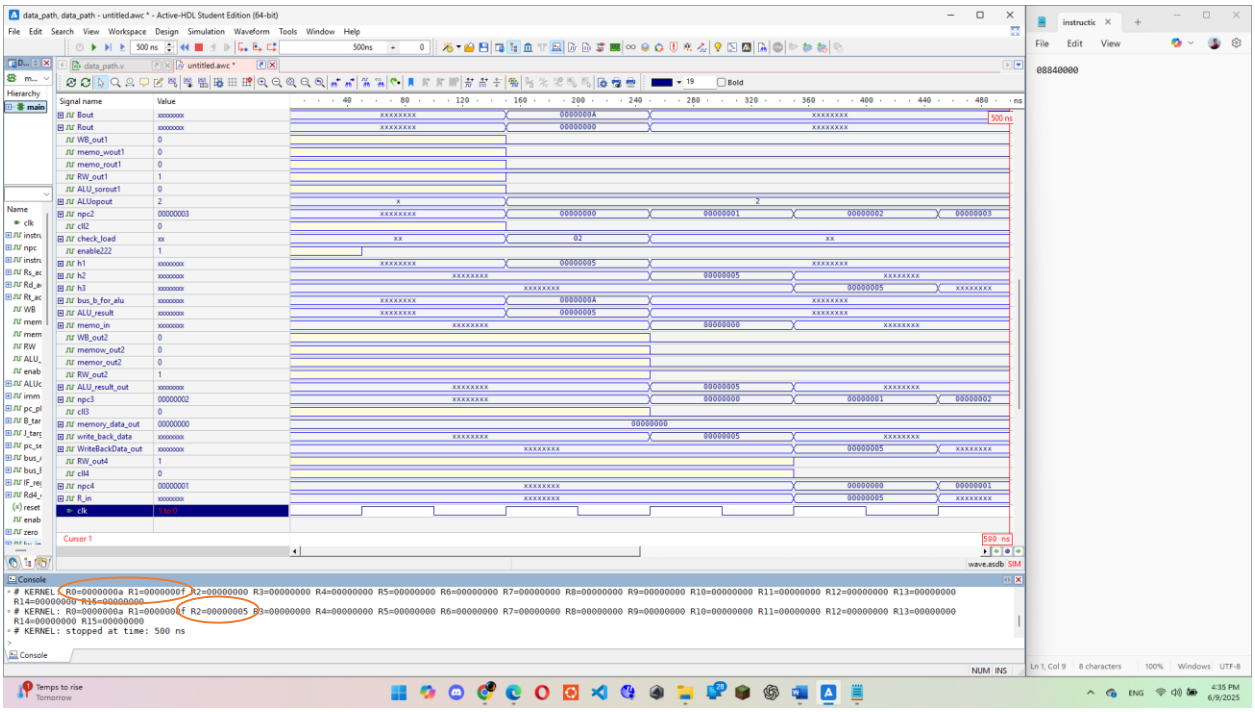


Figure 25: EX SUB

A SUB instruction that subtracts R1 from R0 and stores the result in R2.

CMP

CMP R2, R0,R1

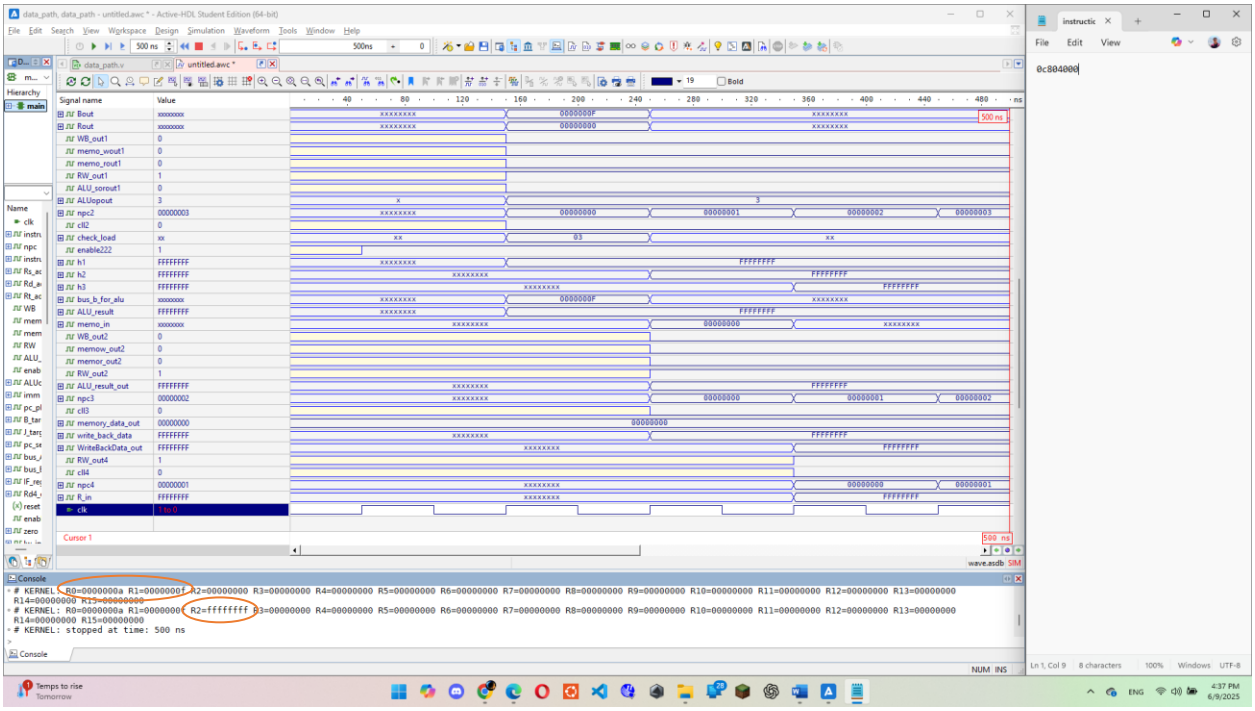


Figure 26: EX CMP1

A CMP instruction that compares Rs and Rt, and stores -1 in R2 if Rs < Rt. In our example, Rs = a and Rt = f, it stores -1 in R2.

CMP R2, R1, R0

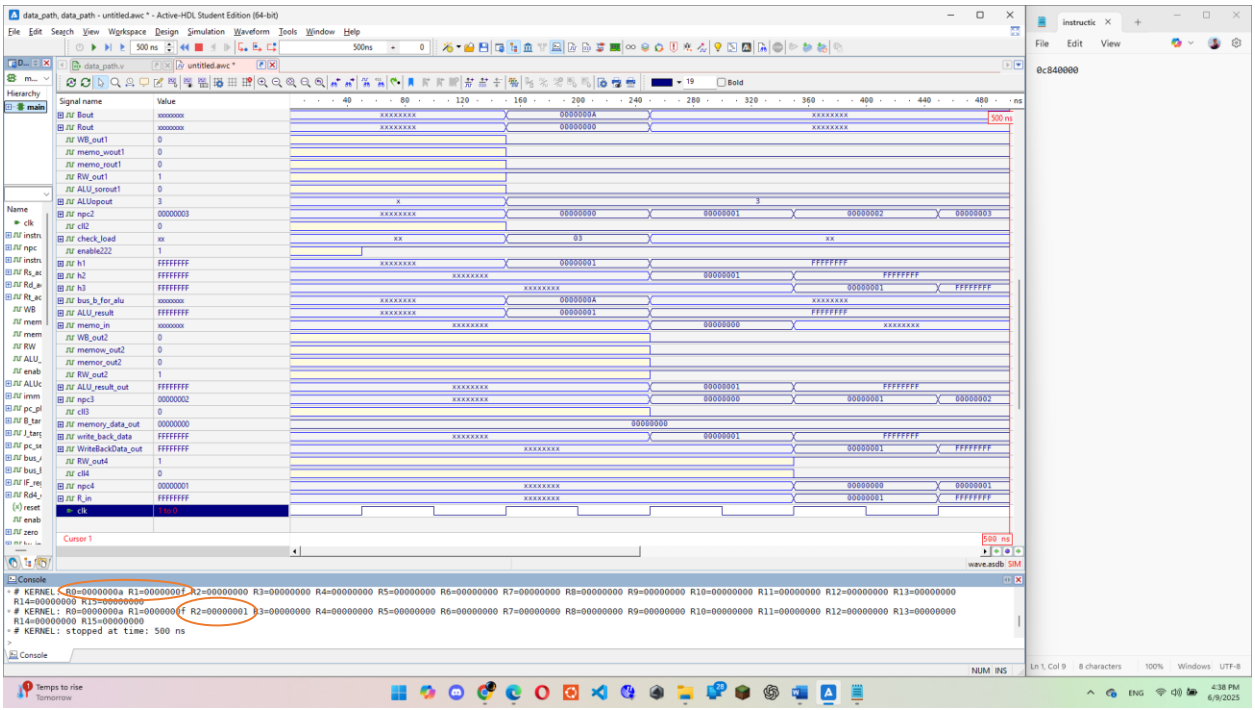


Figure 27: EX CMP2

A CMP instruction that compares Rs and Rt, and stores 1 in R2 if Rs > Rt. In our example, Rs = f and Rt = a, it stores 1 in R2.

CMP R2, R1 , R0

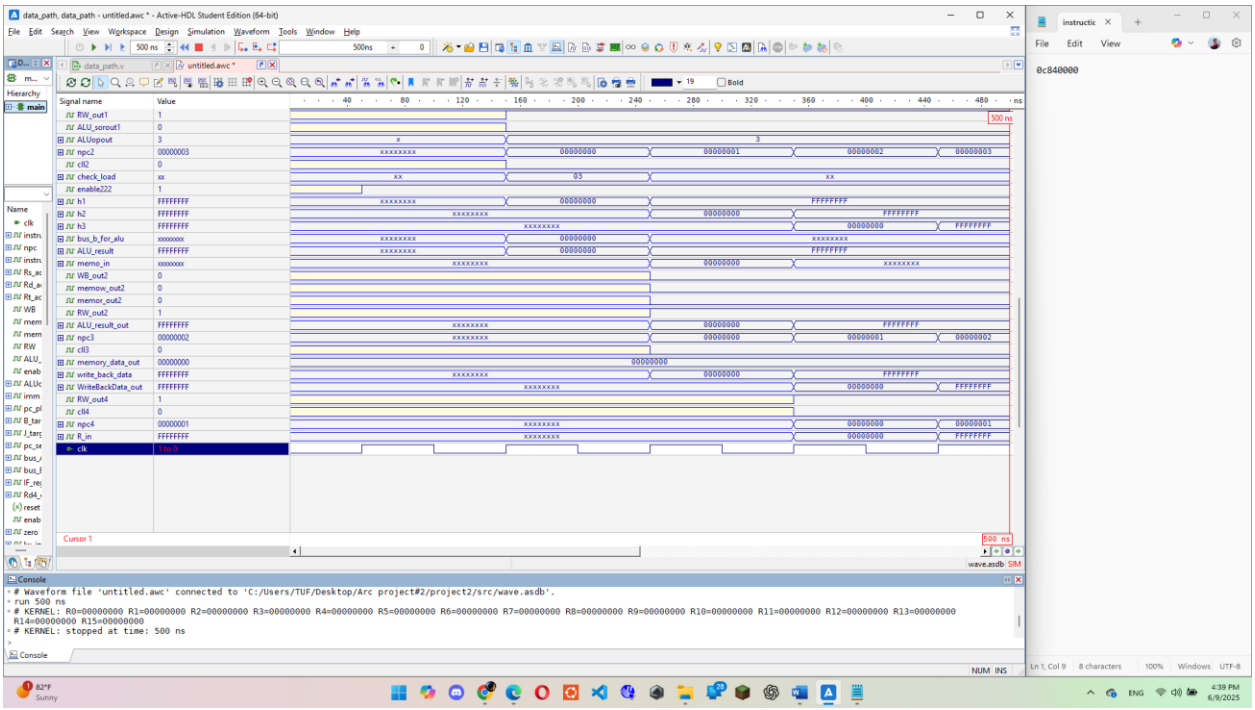


Figure 28: EX CMP3

A CMP instruction that compares Rs and Rt, and stores 0 in R2 if Rs == Rt. In our example, Rs = 0 and Rt = 0, it stores 0 in R2.

ORI

ORI R2,R0,a

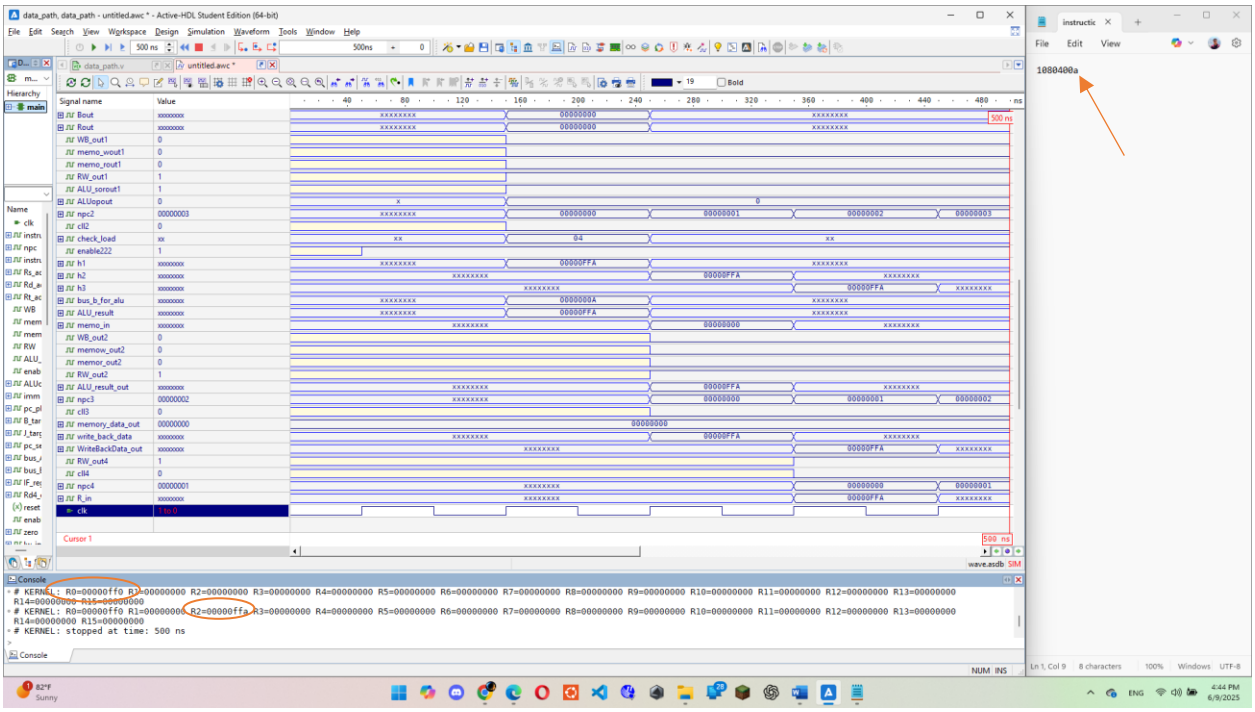


Figure 29: EX ORI

An OR instruction that performs a bitwise OR operation between Rs and imm, and stores the result in Rd. In our example, we do the or op. between R0 and imm = a, then store result in R2.

ADDI

ADDI R2, R0,a

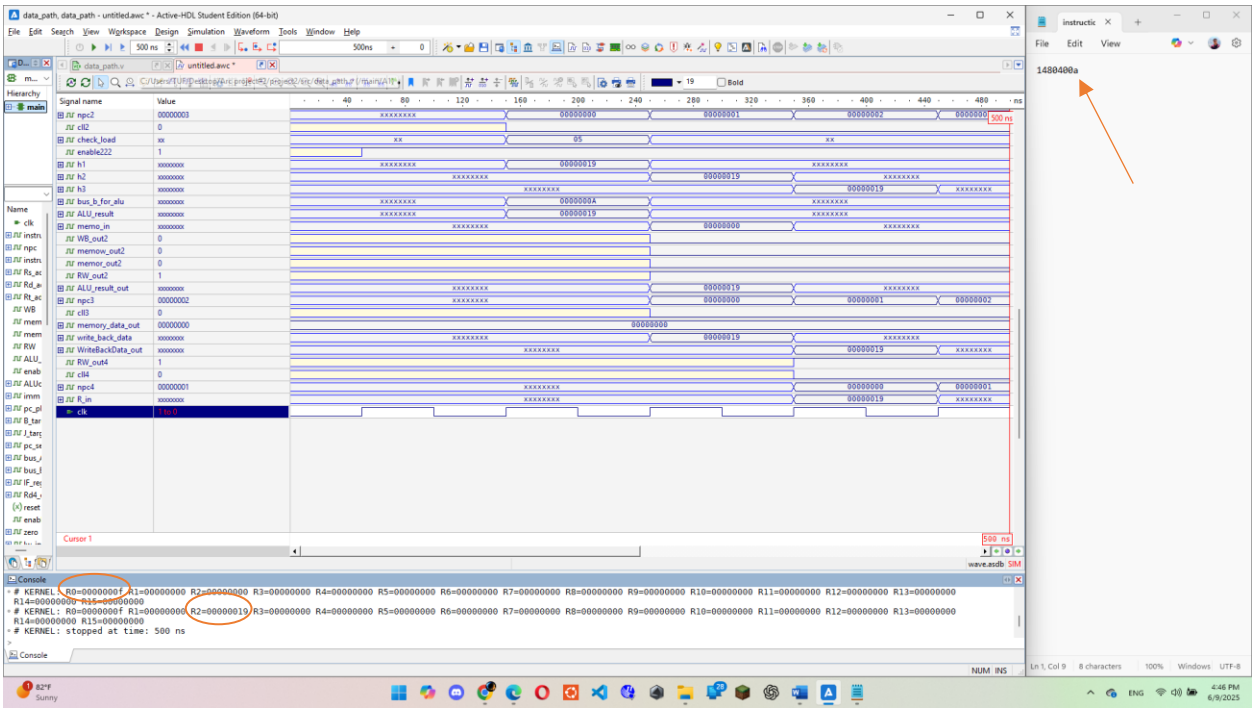


Figure 30: EX ADDI

An ADD instruction that performs an addition of Rs and imm, and stores the result in Rd. In our example, we do the add op. between R0 and imm = a, then store result in R2

LW

LW R0,0(R0)

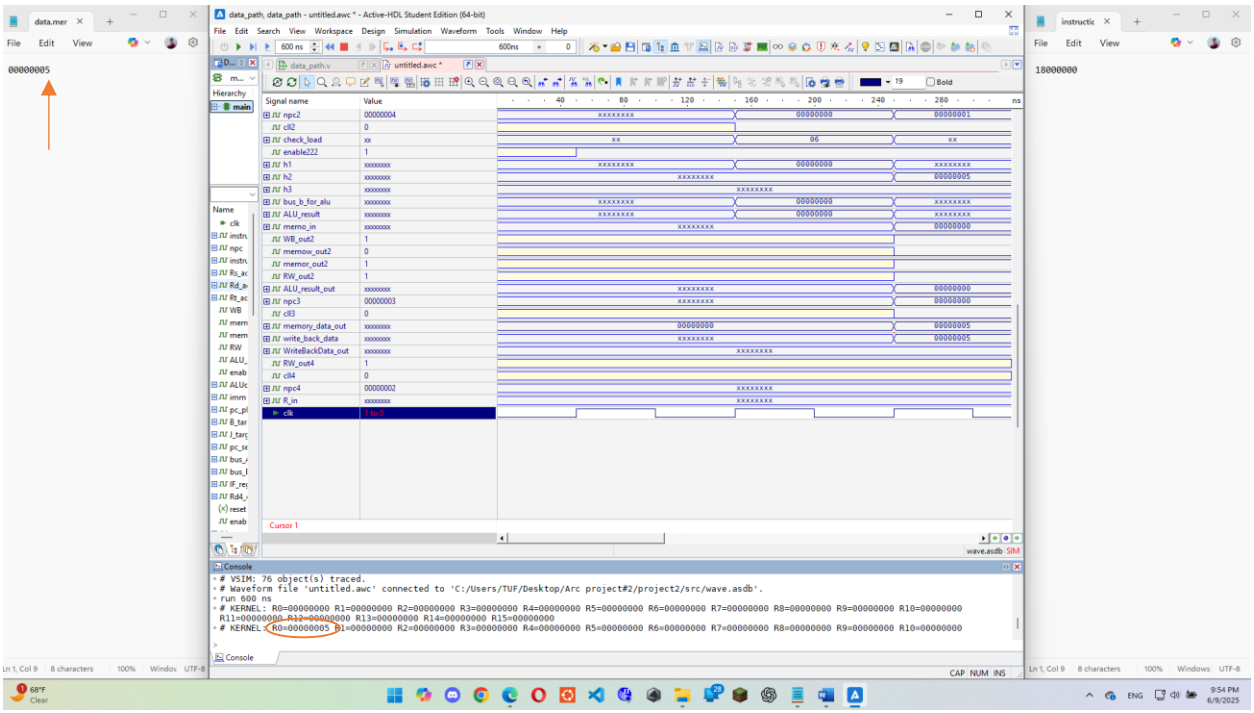


Figure 31: EX LW

Load data from data memory where the Rs is the base address and imm is displacement then store data in Rd. in our example Rs = R0 and imm = 0 then we store 5 in Rd which is also R0.

SW

SW R0,0(R1)

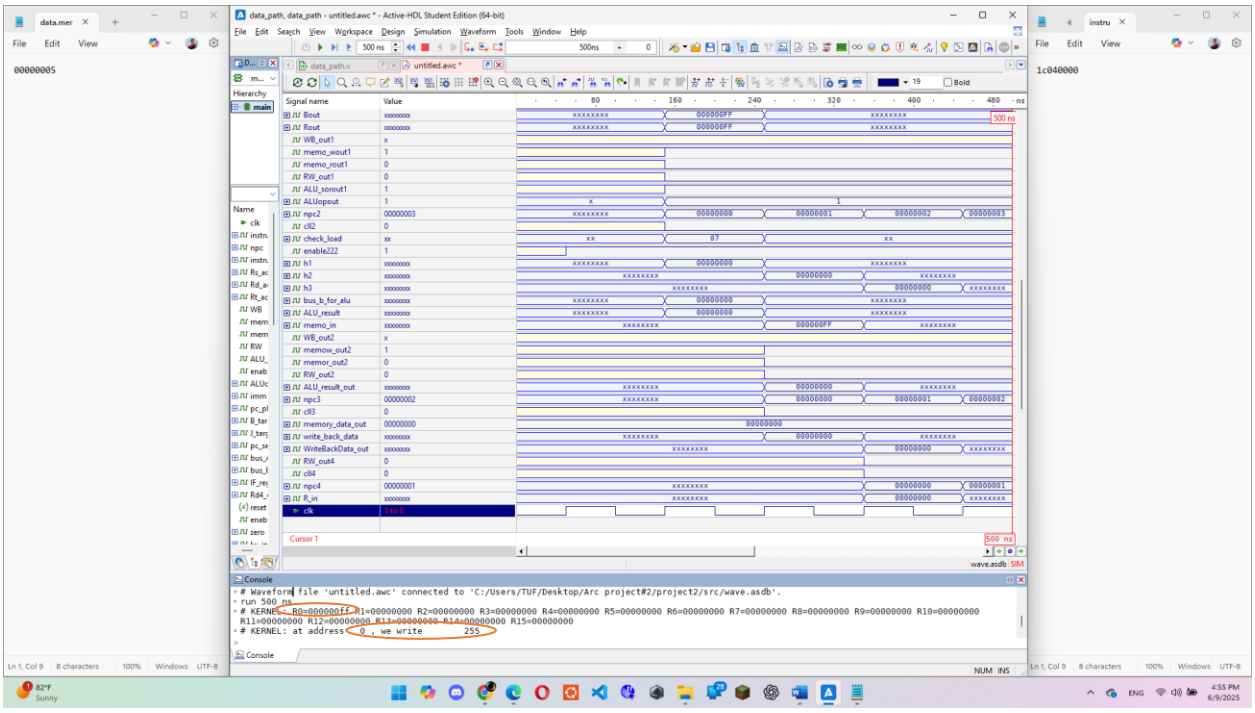


Figure 32: EX SW

store data to data memory where the Rs is the base address and imm is displacement and stored data is Rd. in our example Rs = R0 and imm = 0 then we store ff in address =0 .

the result in the figure is the shown in the decimal.

LDW

LWD R0,0(R2)

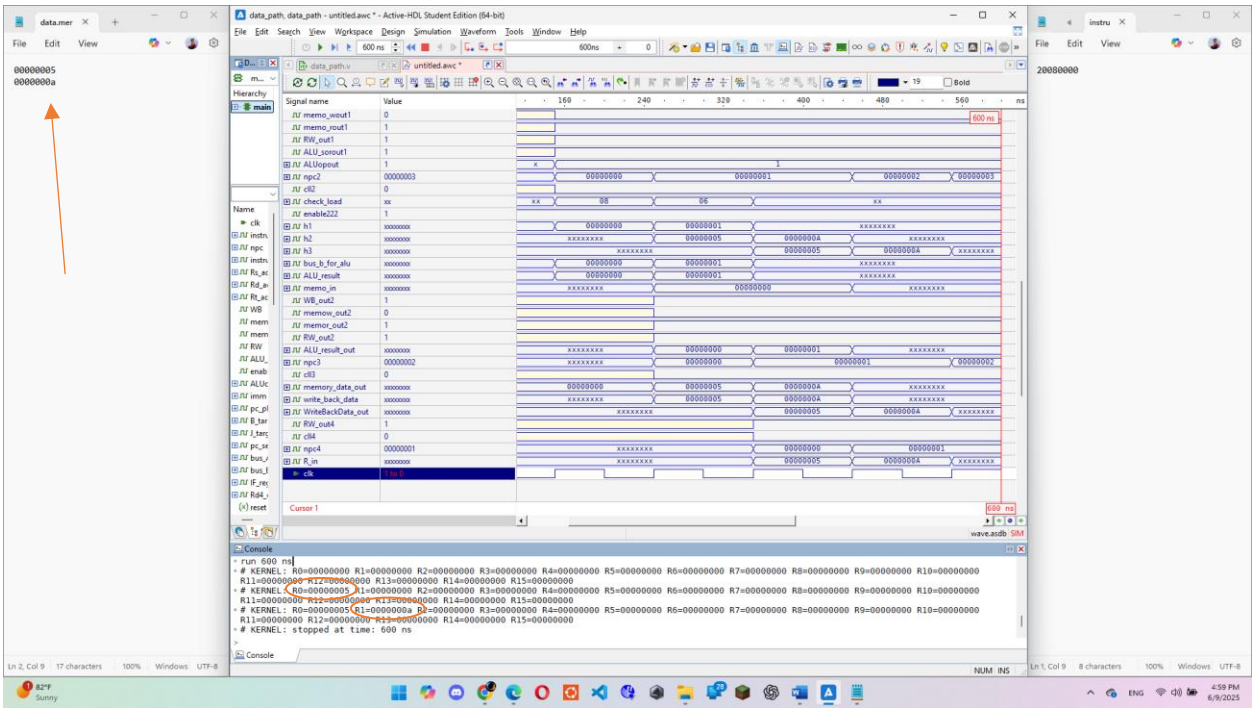


Figure 33: EX LDW

The Double Load instruction loads two consecutive values from data memory. The register Rs is used as the base address, and imm is the initial displacement. The first value is loaded from the address Rs + imm and stored in Rd, which is the first destination register. Then, using imm + 1 as the second displacement, the next value is loaded from memory and stored in Rd + 1, the second destination register. Throughout this operation, Rs remains unchanged. For example, if Rs = R0 and imm = 0, the first loaded value is stored in Rd = R0, and the second value is stored in R1 (i.e., R0 + 1). If the value at memory[R0 + 0] is 5, then R0 will be set to 5, and R1 will receive the value at memory[R0 + 1].

SDW

SDW R0,0(R2)

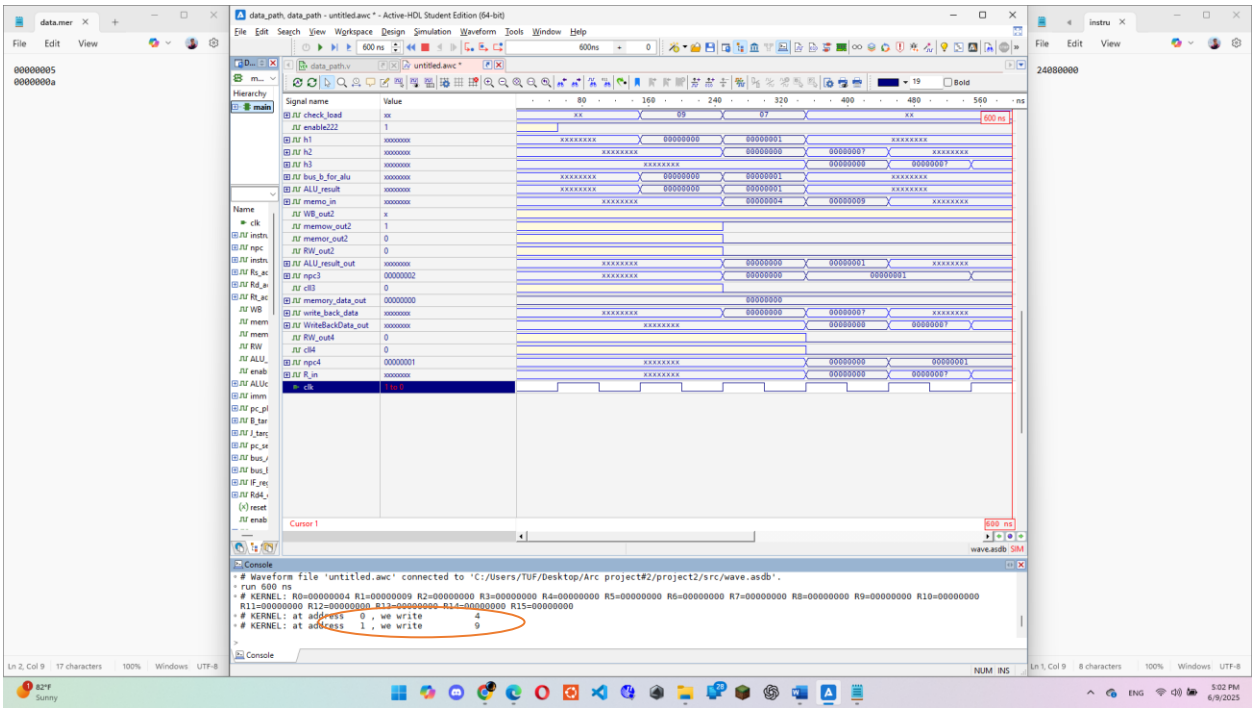


Figure 34: EX SDW

The Double Store instruction stores two consecutive register values into data memory. The register Rs is used as the base address, and imm is the initial displacement. The first value, from register Rd, is stored at the memory address Rs + imm. Then, using imm + 1 as the second displacement, the value from Rd + 1 is stored at the address Rs + imm + 1. Throughout this operation, Rs remains unchanged. For example, if Rs = R0, imm = 0, and Rd = R0, then the value in R0 is stored at memory[R2 + 0], and the value in R3 (i.e., Rd + 1) is stored at memory[R2 + 1].

BZ

BZ R0, a

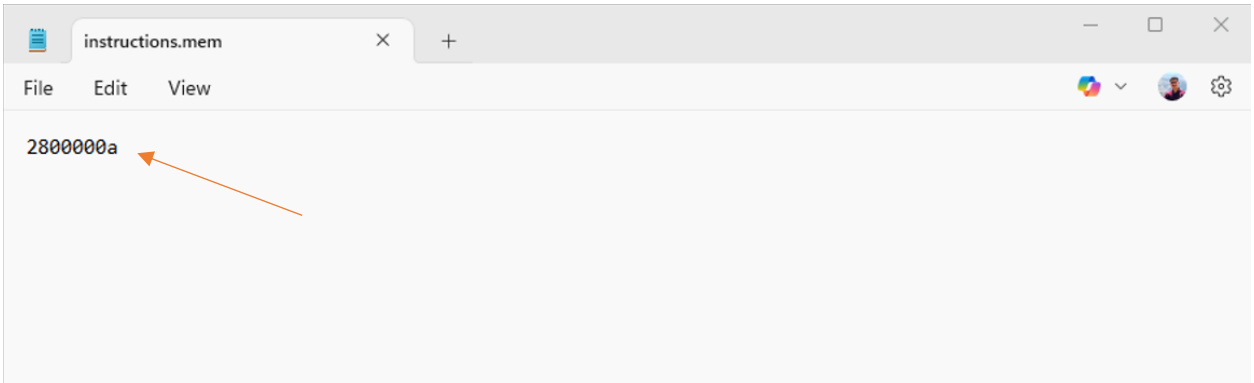


Figure 35: EX BZ instruction memo

Branch target address (pc + ex _ imm)

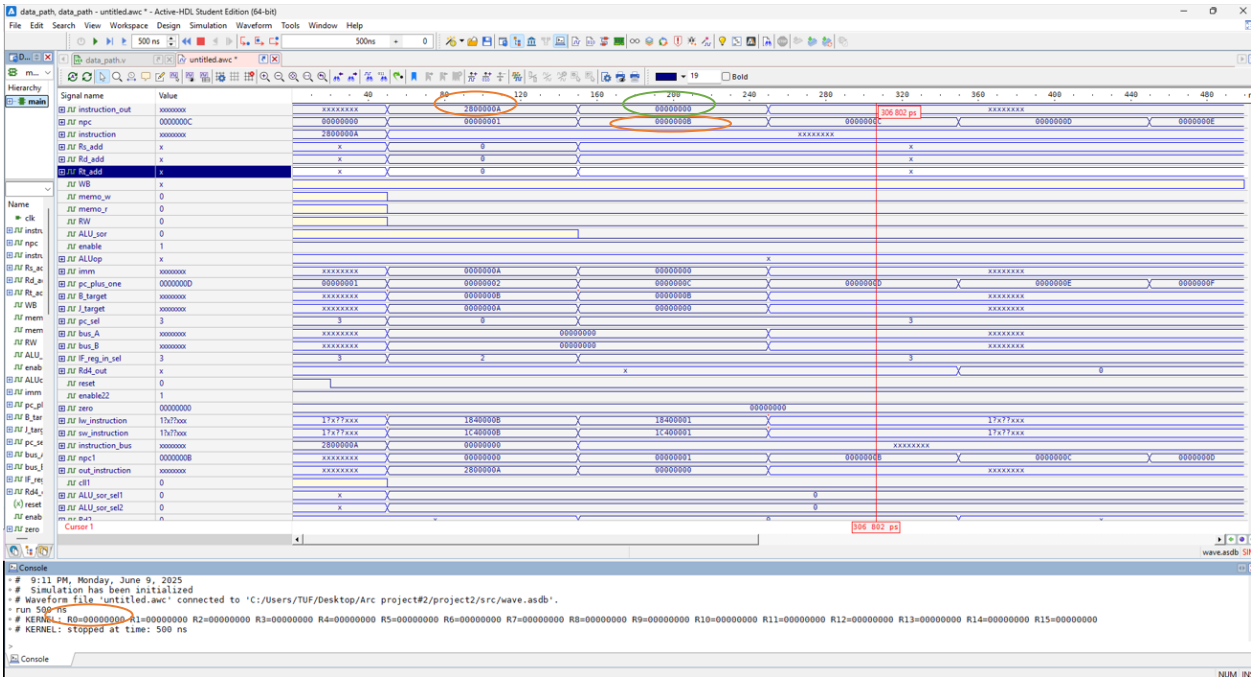


Figure 36: EX BZ branch

In this case branch jump to BTA since the value of Rs which is R0 = 0 . so we pass the stall cycle and pc become = pc + imm (1+a) .

No branch

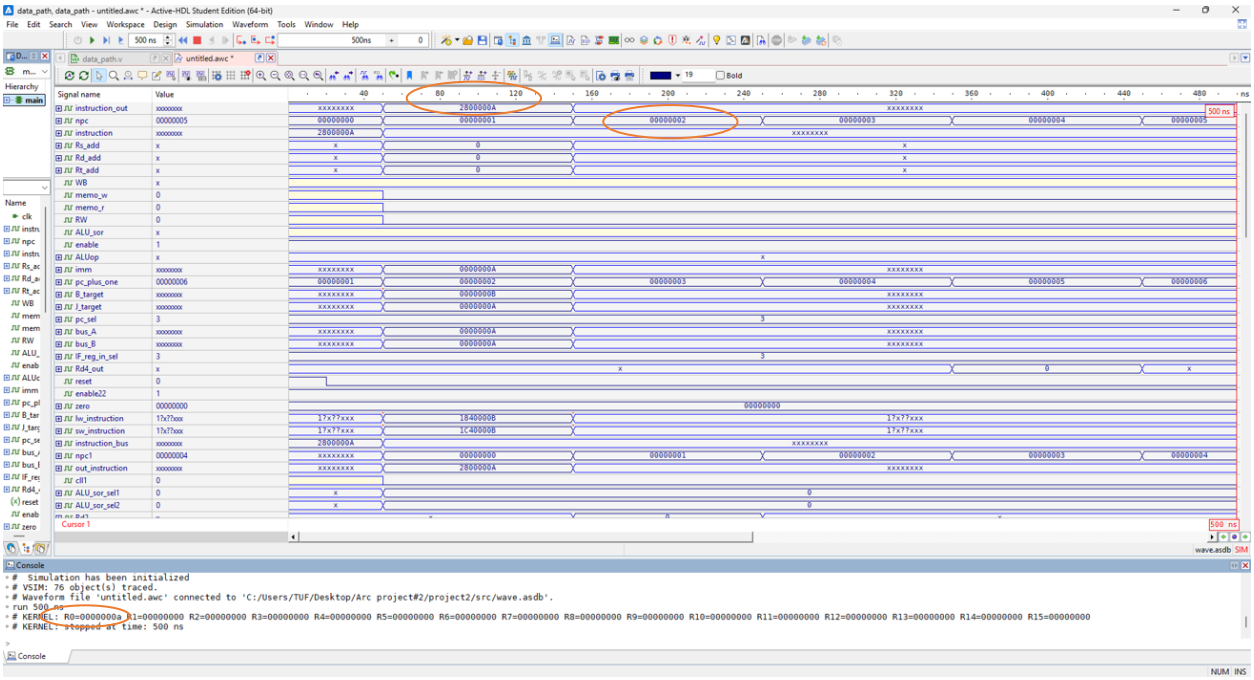


Figure 37: EX BZ no branch

In this example, the Branch instruction doesn't make sense because the value of Rs (which is R0) is not equal to zero.

BGZ

BGZ R0, a



Figure 38: EX BGZ instruction memo

Branch target address (pc + ex _ imm)

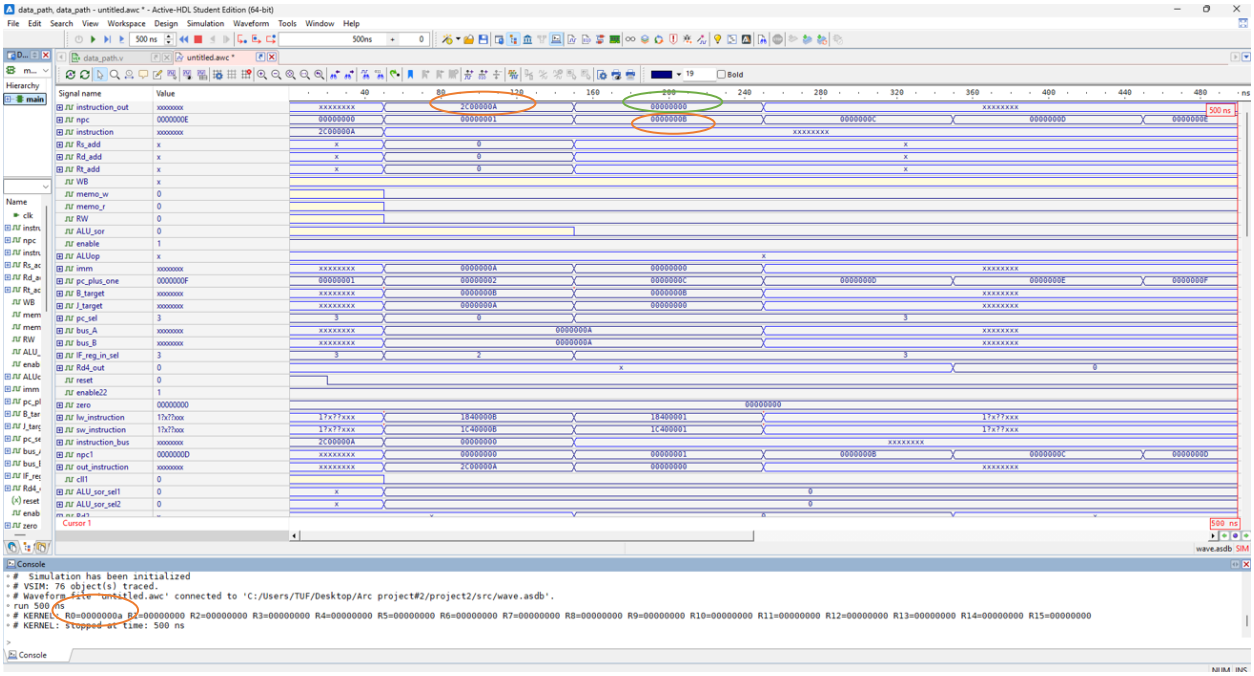


Figure 39: EX BGZ branch

In this case branch jump to BTA since the value of Rs which is R0 > 0. So we pass the stall cycle and pc become = pc + imm (1+a) .

No branch

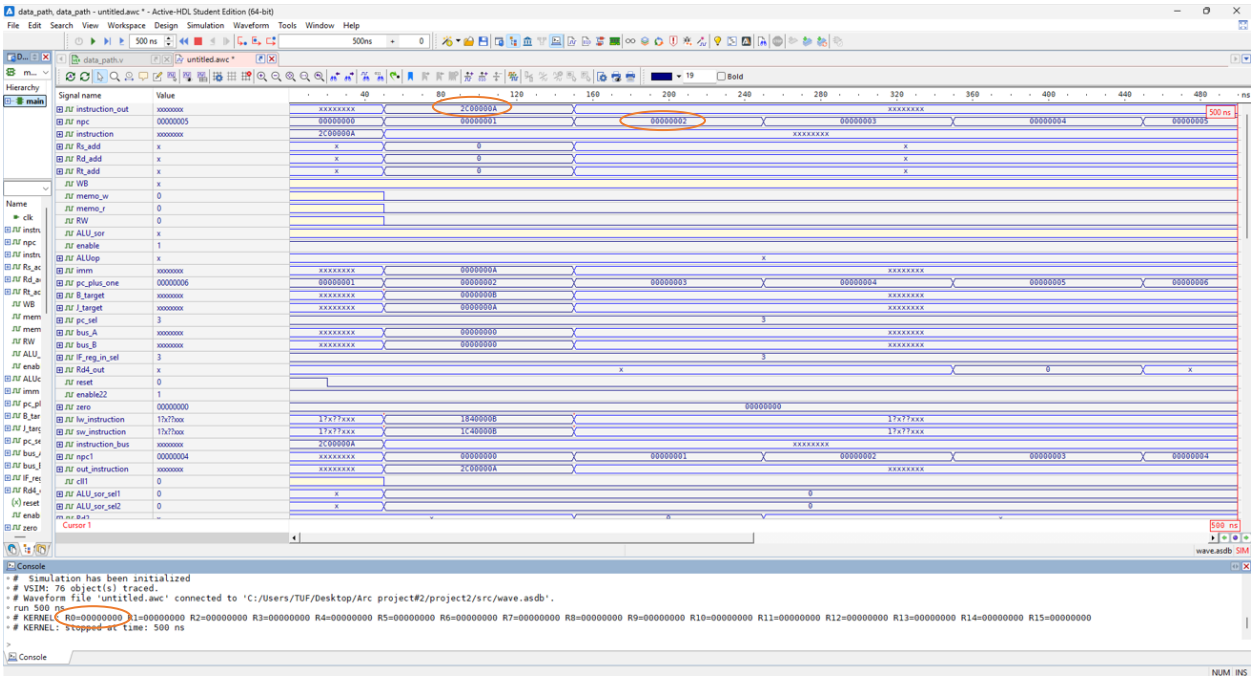


Figure 40: EX BGZ no branch

In this example, the Branch instruction doesn't make sense because the value of Rs (which is R0) isn’t grater than zero .

BLZ

BLZ R0,a

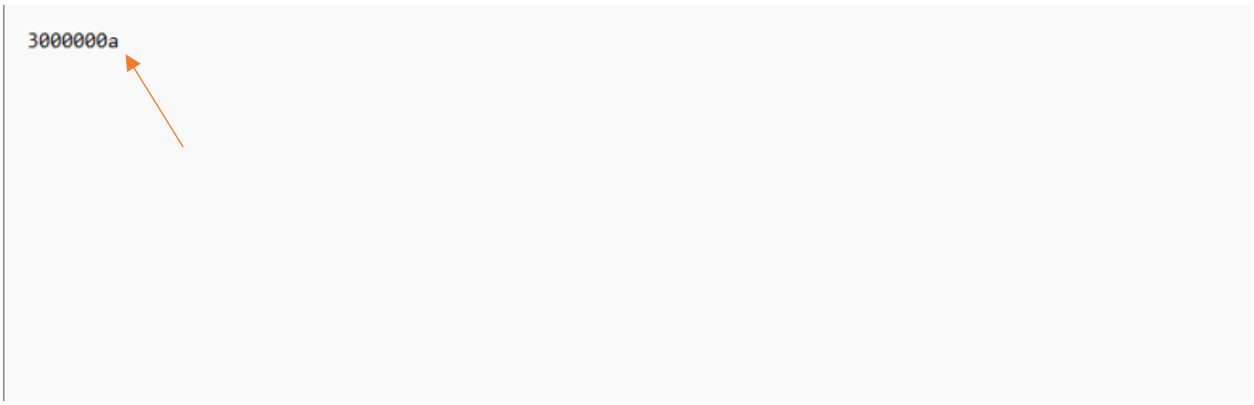


Figure 41: EX BLZ instruction memo

Branch target address (pc + ex _ imm)

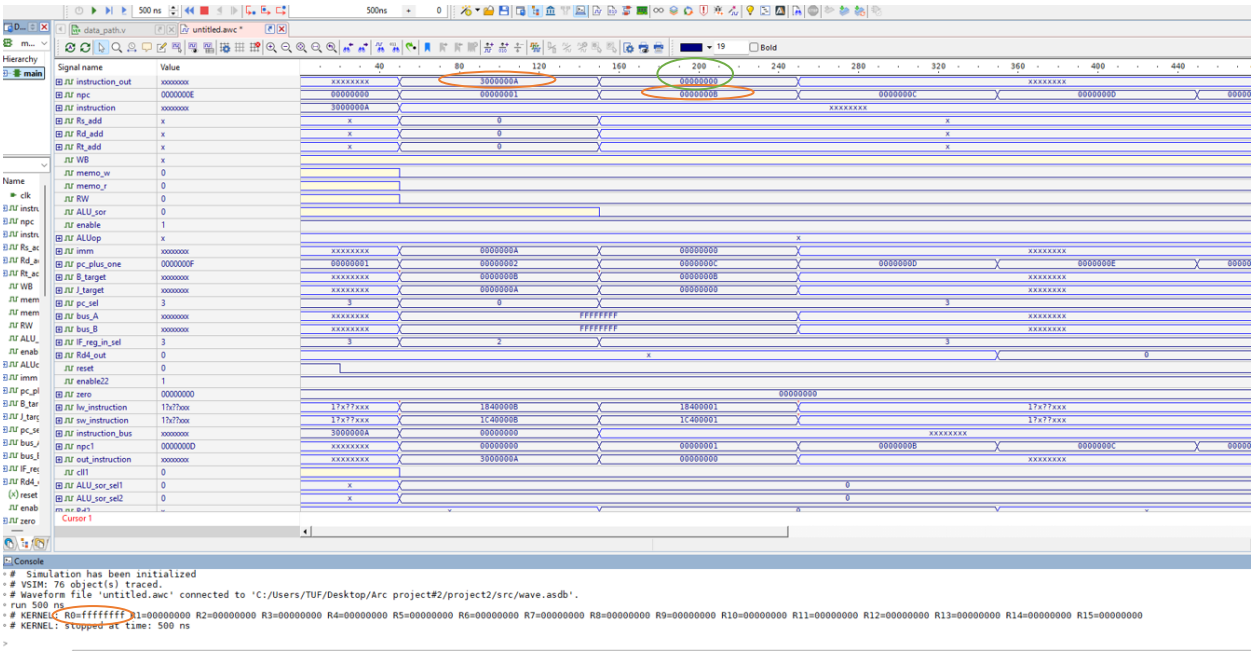


Figure 42: EX BLZ branch

In this case branch jump to BTA since the value of Rs which is R0 < 0. So we pass the stall cycle and pc become = pc + imm (1+a) .

No branch

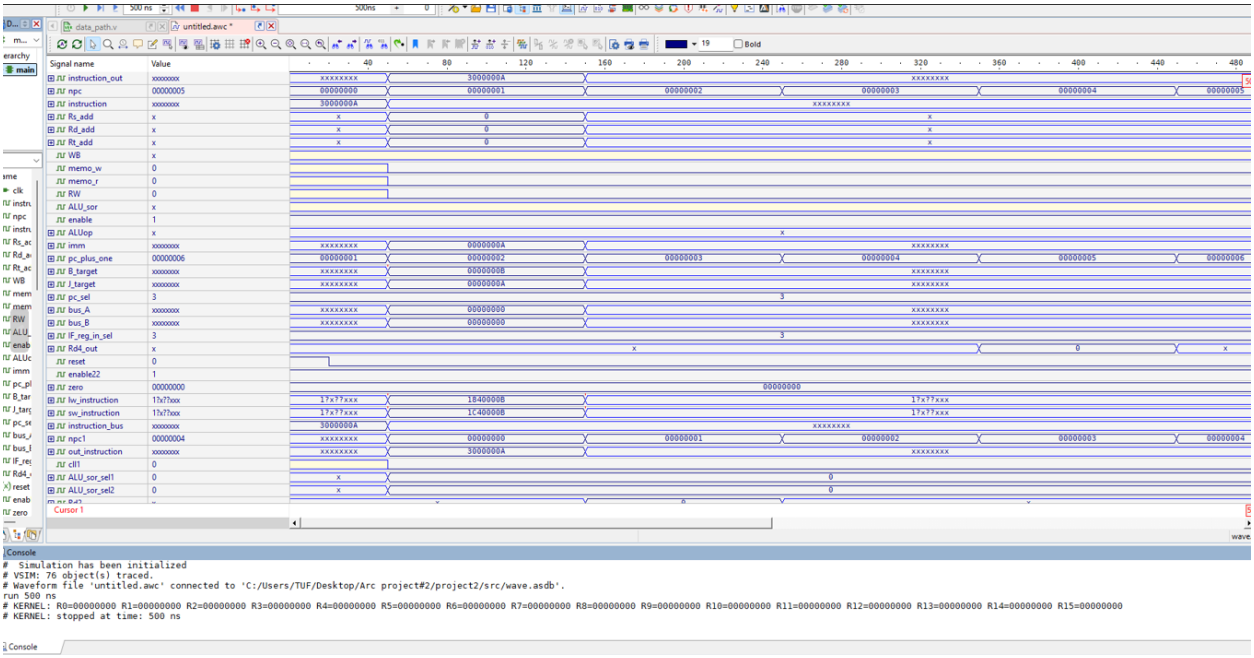


Figure 43: EX BLZ no branch

In this example, the Branch instruction doesn't make sense because the value of Rs (which is R0) isn’t less than zero .

JR

JR , R0

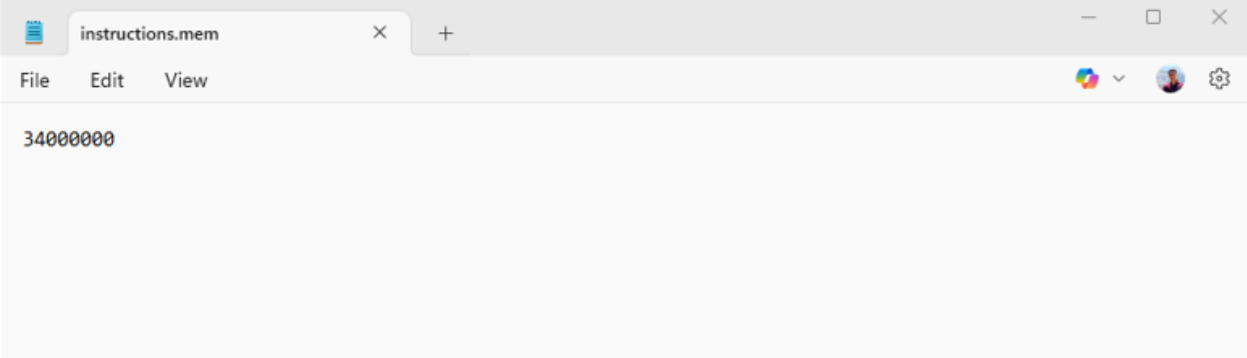


Figure 44: EX JR instruction memo

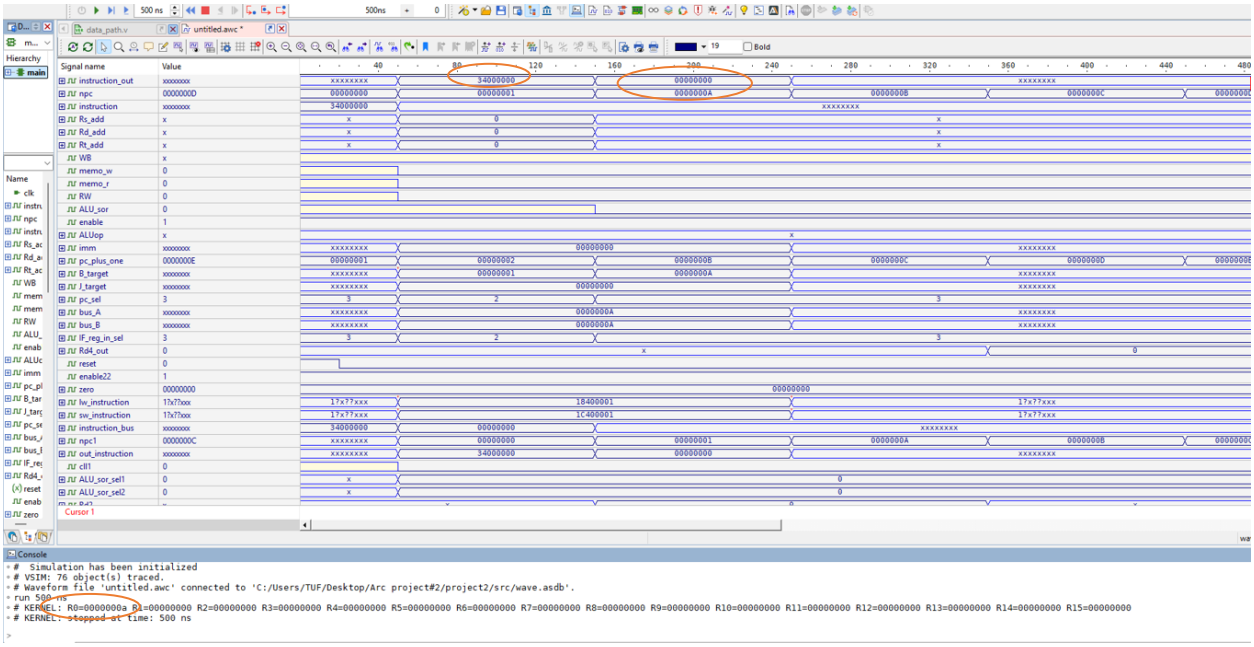


Figure 45: EX JR

The JR instruction uses the value stored in the register Rs to determine the jump target address (JTA) also we pass stall cycle. In this example, the value of Rs is taken from the register file in this case, Rs corresponds to register R0. The program counter (PC) is then set to this value, meaning PC = a, where a is the content of register R0.

J

J, b

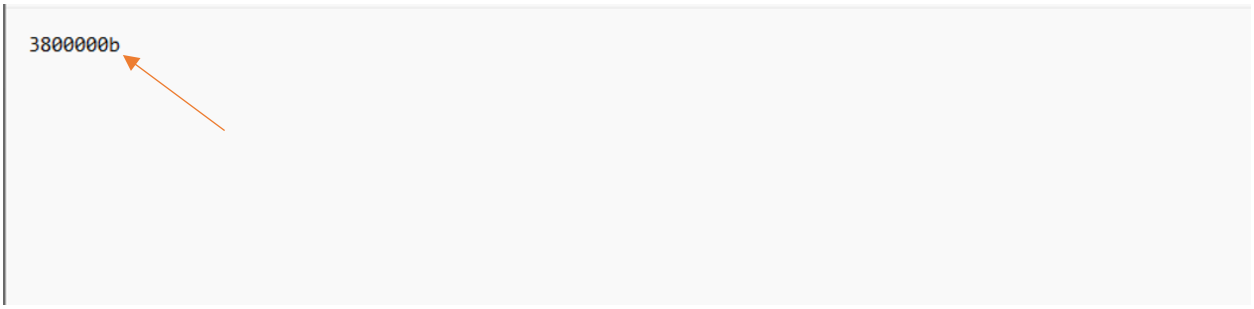


Figure 46: EX J instruction memo

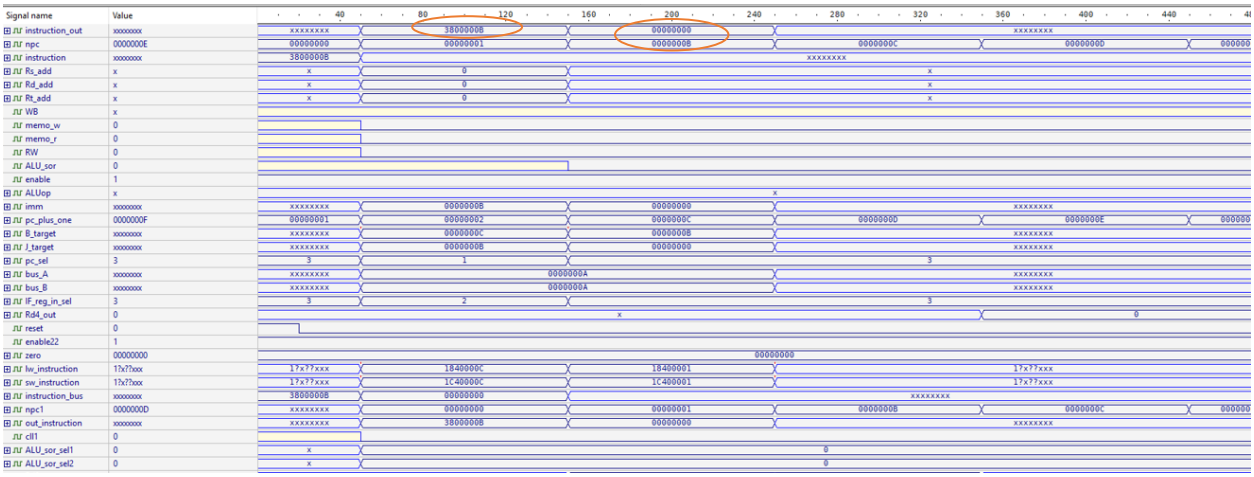


Figure 47: EX J

The J instruction uses the value of imm to determine the jump target address (JTA) also we pass stall cycle. In this example, the value of imm =b. The program counter (PC) is then set to this value, meaning PC = b.

CLL

J, b

R14 = pc.

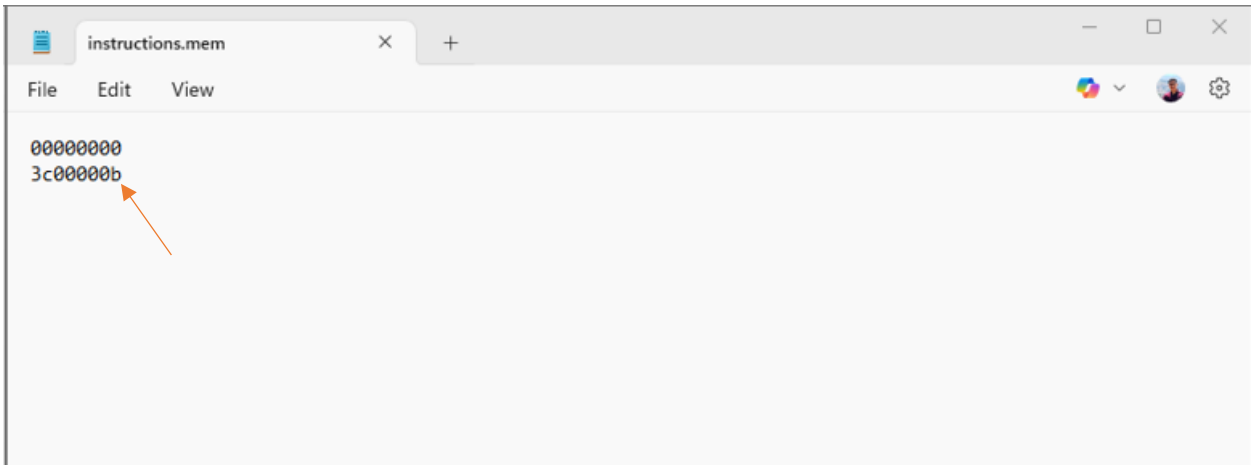


Figure 48: EX CLL instruction memo

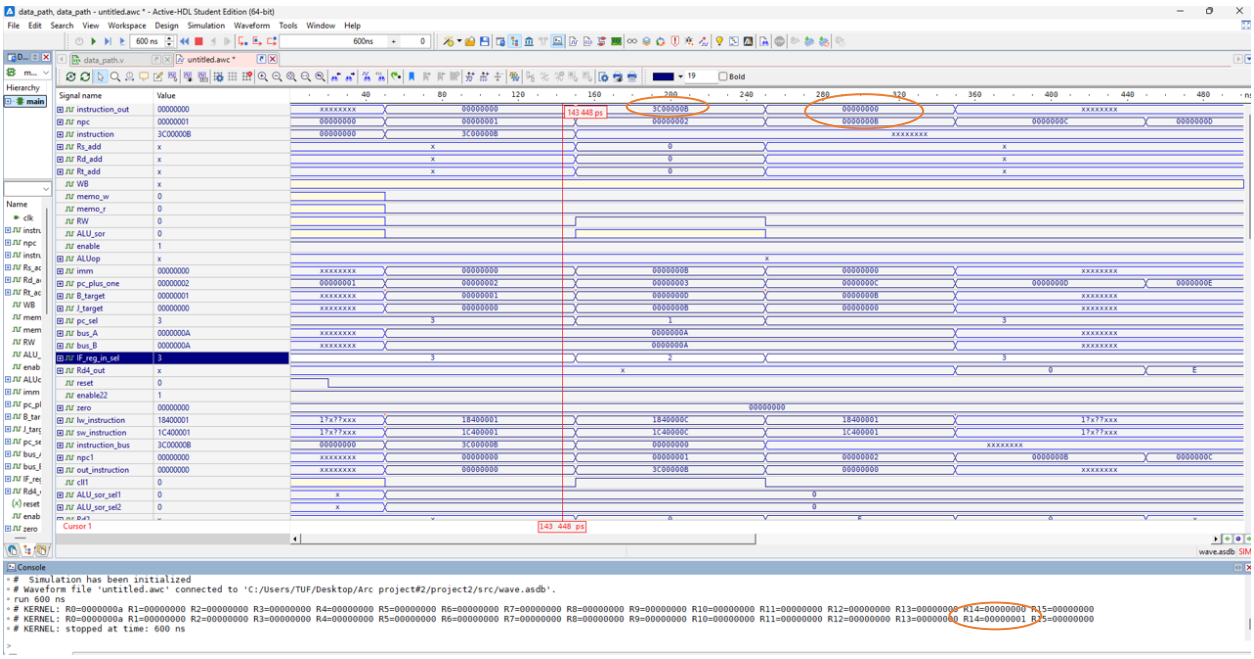


Figure 49: EX CLL

The CLL instruction works similarly to the original jump instruction, as it uses the immediate value (imm) to determine the jump target address (JTA). Additionally, it introduces a stall cycle and stores the return address in register R14.

In this example, the immediate value imm is b. The program counter (PC) is set to this value, so PC = b. Meanwhile, the return address (which is 1 in this case) is stored in register R14 for later use.

Code example

Code 1:

LDW R0, 0(R2)

L1: SUB R0, R0, R1

BZ R0, L2

J L1

L2:

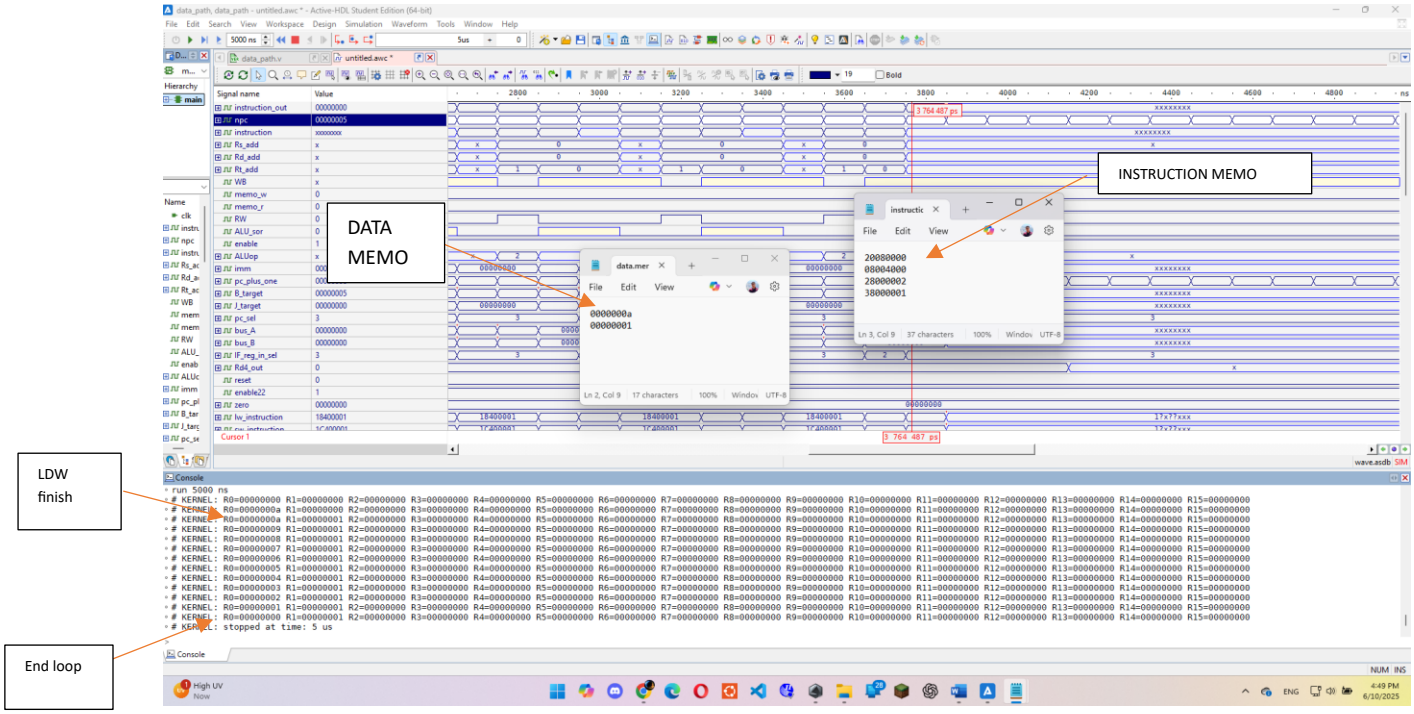


Figure 50: CODE 1

Code 2:

ORI R0, R0, -8

ORI R1, R1, 1

ORI R3, R3 3

L1: ADD R0, R0, R1

BGZ R0, L2

JR R3 L1

L2:

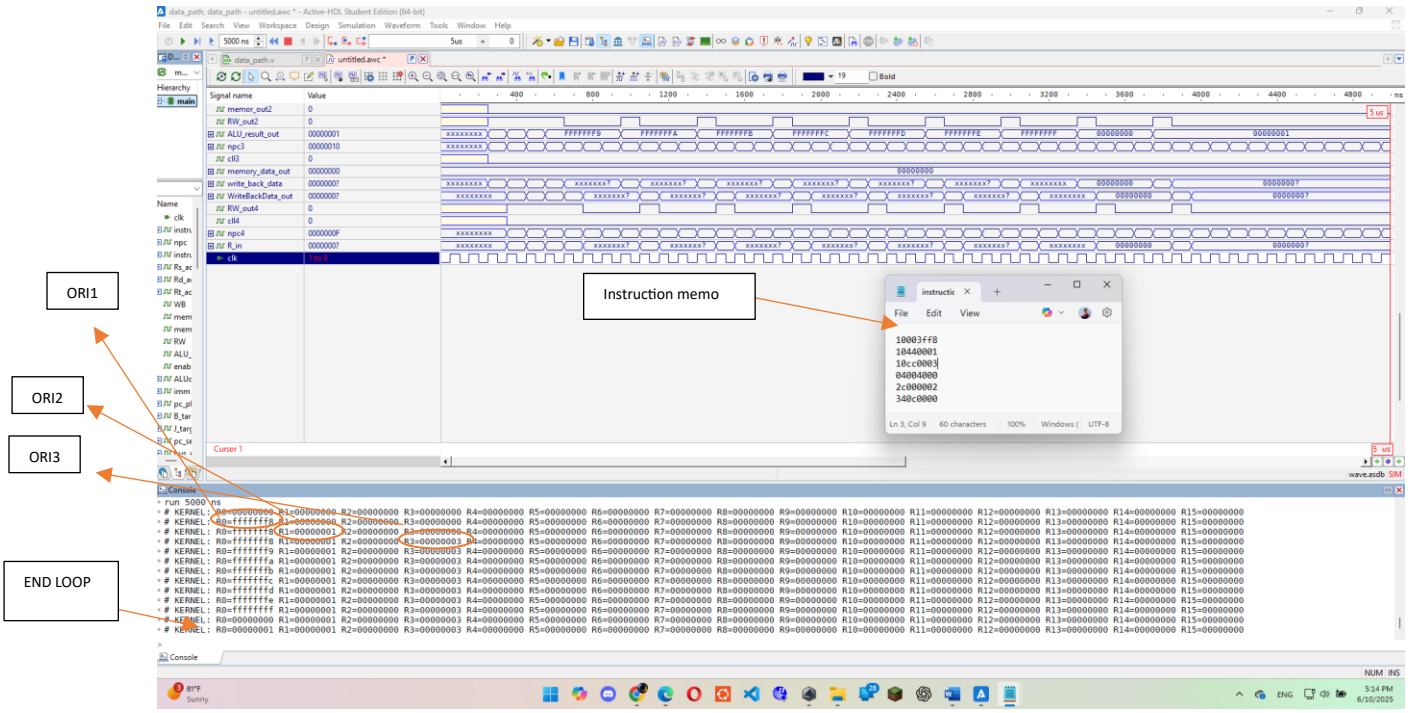


Figure 51: CODE 2

Code 3:

ORI R0, R0, 1

SW R0, 0(R1)

LW R1 ,0(R2)

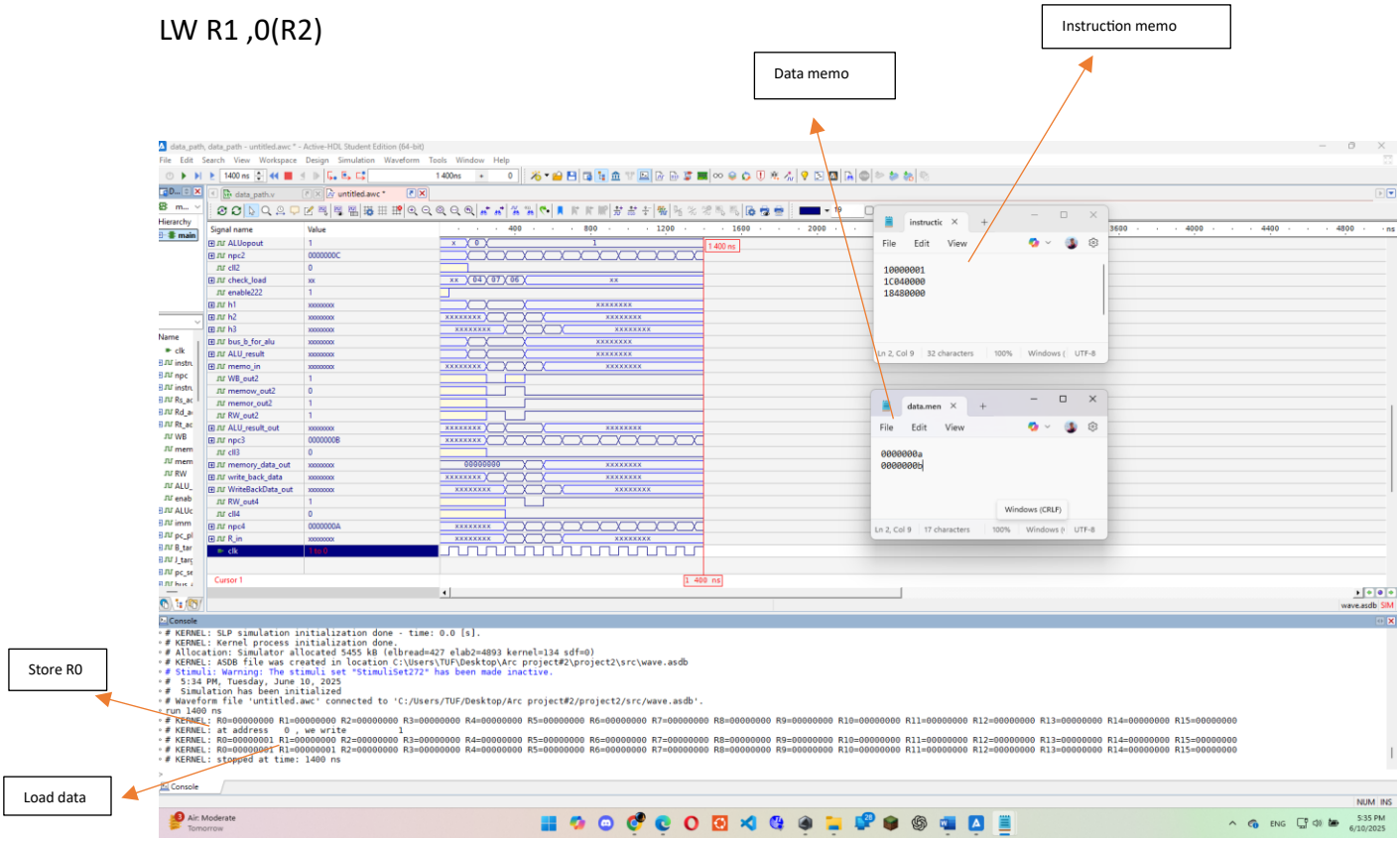


Figure 52: CODE 3

Code 4:

After we assign R2= 3 and R3 = 9 we do the following code

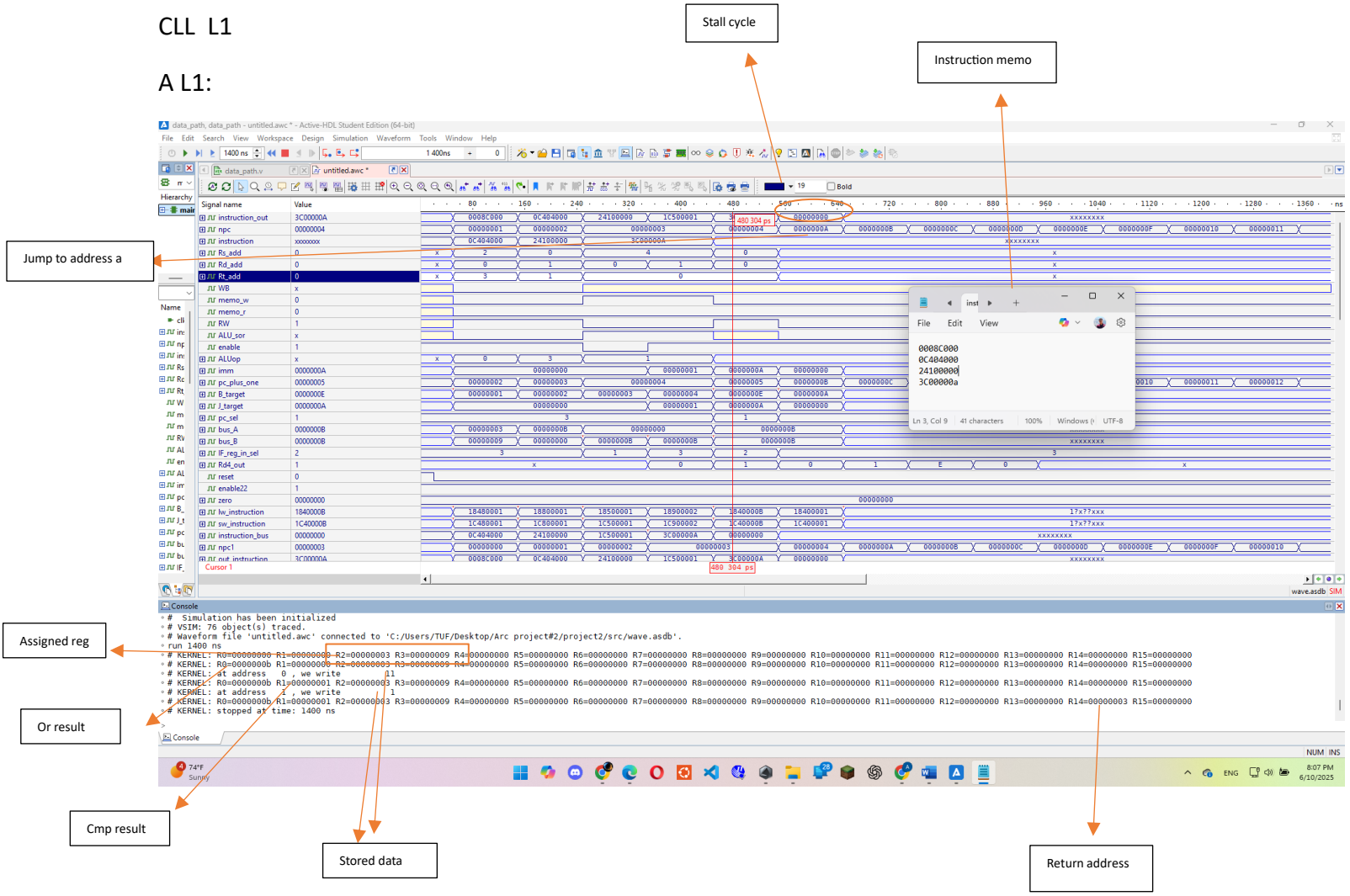
OR R0, R2, R3

CMP R1, R0, R1

SDW R0,0(R4)

CLL L1

A L1:



RTL Pipeline Design Description

R-Type Instructions

(OR, ADD, SUB, CMP — Opcodes: 0 to 3)

Instruction Format: INSTRUCTION Rd, Rs, Rt

Pipeline Stages:

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**: A ← Reg[Rs], B ← Reg[Rt], control signals set based on Opcode
- **EX**:
    - OR: ALUOut ← A | B
    - ADD: ALUOut ← A + B
    - SUB: ALUOut ← A - B
    - CMP:
        - if (A == B) → ALUOut ← 0
        - else if (A < B) → ALUOut ← -1
        - else → ALUOut ← 1
- **MEM**: No operation; values pass through
- **WB**: Reg[Rd] ← ALUOut

I-Type Instructions

(ORI, ADDI — Opcodes: 4, 5)

Instruction Format: INSTRUCTION Rd, Rs, Imm

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**: A ← Reg[Rs], Imm_ext ←
    - Zero-extend if ORI

    - Sign-extend if ADDI
- **EX**:
    - ORI: ALUOut ← A | Imm_ext
    - ADDI: ALUOut ← A + Imm_ext
- **MEM**: Pass-through
- **WB**: Reg[Rd] ← ALUOut

Load/Store Instructions

LW Rd, Imm (Rs) — Opcode: 6

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**: A ← Reg[Rs], Imm_ext ← sign-extended Imm
- **EX**: ALUOut ← A + Imm_ext
- **MEM**: MDR ← DMEM[ALUOut]
- **WB**: Reg[Rd] ← MDR

SW Rd, Imm(Rs) — Opcode: 7

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**: A ← Reg[Rs], B ← Reg[Rd], Imm_ext ← sign-extended Imm
- **EX**: ALUOut ← A + Imm_ext
- **MEM**: DMEM[ALUOut] ← B
- **WB**: No write-back

Double-Word Instructions

LDW Rd, Imm(Rs) — Opcode: 8

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**:
    - if (Rd[0] == 1) → Exception
    - A ← Reg[Rs], Imm_ext ← sign-extended Imm
    - Stall next cycle to handle second word
- **EX**: ALUOut ← A + Imm_ext
- **MEM** (Cycle 1): MDR1 ← DMEM[ALUOut]
- **MEM** (Cycle 2): MDR2 ← DMEM[ALUOut + 1], PC frozen
- **WB**: Reg[Rd] ← MDR1, Reg[Rd + 1] ← MDR2

SDW Rd, Imm(Rs) — Opcode: 9

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**:
    - if (Rd[0] == 1) → Exception
    - A ← Reg[Rs], Data1 ← Reg[Rd], Data2 ← Reg[Rd+1], Imm_ext ← sign-extended Imm
    - Stall next cycle to complete store
- **EX**: ALUOut ← A + Imm_ext
- **MEM** (Cycle 1): DMEM[ALUOut] ← Data1
- **MEM** (Cycle 2): DMEM[ALUOut + 1] ← Data2, PC frozen
- **WB**: No write-back

Branch Instructions

BZ Rs, Label — Opcode: 10

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**: A ← Reg[Rs], Branch_Target ← PC + sign-extended Imm
- **EX**:
    - if (A == 0) → PC ← Branch_Target
    - else → PC remains

BGZ Rs, Label — Opcode: 11

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**: A ← Reg[Rs], Branch_Target ← PC + sign-extended Imm
- **EX**:
    - if ($signed(A) > 0) → PC ← Branch_Target
    - else → PC remains

BLZ Rs, Label — Opcode: 12

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**: A ← Reg[Rs], Branch_Target ← PC + sign-extended Imm
- **EX**:
    - if ($signed(A) < 0) → PC ← Branch_Target
    - else → PC remains

Jump Instructions

JR Rs — Opcode: 13

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**: PC ← Reg[Rs]
- **EX/MEM/WB**: Flushed

J Label — Opcode: 14

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**: PC ← PC + sign-extended Imm
- **EX/MEM/WB**: Flushed

CLL Label — Opcode: 15

- **IF**: IR ← IMEM[PC], PC ← PC + 1
- **ID**: Reg[R14] ← PC, PC ← PC + sign-extended Imm
- **EX/MEM/WB**: Flushed

Conclusion

The journey of designing and implementing this 5-stage pipelined RISC processor in Verilog has been both challenging and deeply rewarding. Starting from the foundational ISA specifications to the final verification tests, every phase demanded meticulous attention to detail and iterative problem-solving. The result is a functional processor capable of executing arithmetic, logical, memory, and control operations while adhering to the principles of pipelined execution—maximizing throughput without compromising correctness.

Future Improvements

While the processor meets all functional requirements, future work could explore:

- Dynamic branch prediction to further reduce control hazards,
- Cache integration for higher memory efficiency,
- Exception handling for invalid operations (e.g., odd LDW registers).

In closing, this project was more than an academic exercise—it was a microcosm of real-world RISC design. The satisfaction of seeing a simulated CALL instruction correctly push a return address to R14, or a pipelined ADD-OR sequence execute flawlessly, underscored the beauty of digital systems: complexity tamed by careful design. As we hand off this report, we carry forward not just a working processor, but a deeper appreciation for the invisible machinery powering modern computing.